

Orquestador de Seguridad: Fuzzing Automatizado de Aplicaciones Web

UNIVERSIDAD TECNOLÓGICA NACIONAL – FACULTAD REGIONAL MENDOZA

Tecnicatura Universitaria en Programación

Subtítulo: Sistema de orquestación para análisis automatizado de vulnerabilidades web mediante fuzzing y escaneo activo

Autores: Tomas Mastropietro, Cristian Krahulik, Juan Segura

Directores: Alberto Cortez y Ariel Enferrel

Línea de Investigación: Seguridad informática · Ciberseguridad ofensiva · Automatización de pruebas · Ingeniería de software

Fecha: 2026

Resumen

El presente trabajo describe el diseño, implementación y validación de un sistema automatizado de análisis de seguridad para aplicaciones web basado en técnicas de fuzzing y escaneo activo. El sistema desarrollado, denominado Orquestador de Seguridad, fue implementado íntegramente en Python y permite coordinar la ejecución de múltiples herramientas especializadas de análisis de vulnerabilidades de manera automatizada y secuencial.

El sistema integra las herramientas OWASP ZAP, ffuf y SQLMap, cada una ejecutándose en entornos contenerizados mediante Docker. El orquestador central se encarga de coordinar la ejecución de cada herramienta, recolectar sus resultados, procesarlos mediante parsers especializados y consolidarlos en un único informe estructurado en formato JSON. La arquitectura implementada se basa en un modelo modular donde cada componente posee una responsabilidad específica: ejecución de herramientas externas, recolección de datos, procesamiento de resultados y consolidación final de hallazgos. Sobre esta base, el sistema se extendió hacia una arquitectura de servicio con API REST asíncrona, persistencia de historial, caché incremental, integración con n8n y un frontend en React.

Las pruebas del sistema fueron realizadas sobre DVWA (Damn Vulnerable Web Application), una aplicación web intencionalmente vulnerable utilizada comúnmente para investigación y entrenamiento en seguridad informática. Durante las pruebas se logró detectar múltiples vulnerabilidades relacionadas con el OWASP Top 10, incluyendo inyecciones SQL, configuraciones de seguridad incorrectas y rutas administrativas expuestas.

Los resultados obtenidos demuestran que la automatización del proceso de fuzzing y escaneo permite reducir el esfuerzo de análisis manual, facilitando la identificación temprana de vulnerabilidades en aplicaciones web. Estas comparaciones se detallan en el capítulo 14, junto con las limitaciones metodológicas correspondientes.

Palabras clave: Fuzzing, Seguridad Web, OWASP ZAP, SQLMap, Automatización, Docker, Python.

Abstract

This work presents the design, implementation and validation of an automated security analysis system for web applications based on fuzzing techniques and active vulnerability scanning. The developed system, called Security Orchestrator, was implemented entirely in Python and coordinates the execution of multiple specialized security tools in an automated and sequential pipeline.

The system integrates OWASP ZAP, ffuf, and SQLMap, each executed in containerized environments using Docker. The Python orchestrator manages tool execution, collects raw outputs, processes the results through specialized parsers, and consolidates all findings into a single structured JSON report.

The system was tested using DVWA (Damn Vulnerable Web Application) as a controlled laboratory target. The tests successfully identified vulnerabilities associated with the OWASP Top 10, including SQL injection, insecure configurations, and exposed administrative routes.

The results show that automated fuzzing orchestration improves vulnerability discovery efficiency and reduces manual testing effort in web security assessments, with the corresponding measurement caveats detailed in Chapter 14.

Keywords: Fuzzing, Web Security, OWASP ZAP, Automation, Vulnerability Scanning, Docker, Python.

Índice

Resumen	1
Abstract	2
Índice de Figuras	5
Índice de Tablas	5
1. Introducción	6
1.1 Contexto Global y Panorama de Vulnerabilidades Web	6
1.2 La Necesidad de Orquestación: Paradigma SOAR y Enfoque DAST Integrado	6
1.3 Propuesta de Valor, Alcance del Prototipo y Entorno Experimental	7
1.4 Estructura del Documento	7
2. Planteo del Problema	8
2.1 Contextualización: La Fricción entre DAST Tradicional y la Agilidad de CI/CD	8
2.2 Desarticulación Operativa y Silos de Información	8
2.3 Taxonomía Estructurada de Problemas Identificados	9
2.4 Brecha Operativa a Resolver por el Orquestador	9
3. Justificación	10
3.1 Justificación Académica e Integración Formativa en la TUP	10
3.2 Justificación Técnica: Paradigma Shift-Left y Enriquecimiento Cruzado	11
3.3 Justificación Económica y Modelo Cuantitativo de Retorno de Inversión (ROI)	11
3.4 Transferencia Tecnológica y Replicabilidad en PyMEs y Ámbito Educativo	12
4. Objetivos	13
4.1 Objetivo General	13
4.2 Objetivos Específicos	13
5. Preguntas de Investigación	15
6. Hipótesis	17
7. Estado del Arte	18
8. Marco Teórico y Conceptual	20
8.1 Fuzzing	20
8.2 Escaneo de Vulnerabilidades	20
8.3 Contenedores Docker	20
8.4 OWASP Top 10	20
8.5 Pipeline de Datos y Arquitectura Modular	20
8.6 Orquestación y Automatización de Seguridad (SOAR)	21
9. Alcances y Limitaciones	22
9.1 Alcances	22
9.2 Limitaciones	23
10. Metodología	25
10.1 Diseño de la Investigación	25
10.2 Fases del Proyecto	25
10.3 Variables	25
10.4 Instrumentos	26
10.5 Amenazas a la Validez	26
10.6 Criterios de Validación	26
11. Arquitectura Propuesta e Implementada	27
11.1 Descripción General	27
11.2 Componentes y Sus Roles	28
11.3 Red Docker y Aislamiento	28
11.4 Flujo de Datos Detallado	29
11.5 Diagrama de Secuencia — Ciclo de Vida de POST /scan	30
11.5.1 Detalle de la Secuencia de Interacción	31
12. Implementación Técnica	33
12.1 Infraestructura Docker	33
12.2 Pipeline de Ejecución	33
12.3 Módulo Runner	33
12.4 Módulo de Escaneo ZAP	33

12.5 Módulo de Escaneo ffuf	33
12.6 Módulo de Escaneo SQLMap	34
12.7 Sistema de Parseo	34
12.8 Consolidación de Resultados	35
12.9 Autenticación Automática	35
12.10 API REST y Frontend	35
13. Resultados Obtenidos	36
13.1 Resumen de Hallazgos	37
13.2 Vulnerabilidades Detectadas por OWASP ZAP	37
13.3 Rutas Descubiertas por ffuf	39
13.4 Análisis Automatizado con SQLMap	39
13.5 Análisis por Categoría OWASP Top 10	40
14. Discusión	41
14.1 Contraste de cobertura: uso individual frente a combinado (H2)	41
14.2 Reducción de esfuerzo manual (H1)	43
14.3 Verificación de H4: impacto del caché incremental	43
14.4 Rendimiento y ajuste de timeouts	45
14.5 Seguimiento longitudinal y sugerencias de mitigación por IA	45
14.6 Comparación con el panorama de la industria	46
15. Conclusiones y Trabajos Futuros	47
15.1 Conclusiones	47
15.1.1 Verificación por objetivo específico	49
15.2 Trabajos Futuros	49
16. Consideraciones Éticas	51
17. Desarrollo Experimental y Validación de Hallazgos	52
17.1 Cronograma y Distribución Temporal del Proceso de Auditoría	52
17.2 Procedimientos de Descubrimiento y Validación Manual por Vector de Ataque	52
17.2.1 Fase de Reconocimiento y Mapeo de Superficie (ZAP Spider y ffuf)	52
17.2.2 Inyección de Comandos del Sistema Operativo (ZAP Active Scan)	53
17.2.3 Vulnerabilidad de Inyección SQL (sqlmap)	53
17.2.4 Cross-Site Scripting Reflejado (ZAP Active Scan)	53
17.2.5 Inclusión Local de Archivos (ZAP Active Scan)	54
17.3 Conclusiones sobre la Metodología de Validación	54
18. Referencias	56
19. Anexos Técnicos	58
Anexo A — Estructura del Proyecto y configuración de docker compose	58
A.1 Estructura del proyecto	58
A.2 Archivo docker-compose.yml	59
A.3 Archivo Dockerfile	61
Anexo B — Fragmento Representativo: Pipeline de Ejecución	62
Anexo C — Fragmento Representativo: Parser de ZAP	63
Anexo D — Ejemplo del JSON Unificado Final (corrida del 5 de agosto de 2026)	64
Anexo E — Fragmento Representativo: Autenticación Automática	65
Anexo F — Fragmento Representativo: API REST	66
Anexo G — Frontend: Panel de Control	67
Anexo H — Diagramas en Tamaño Ampliado	68
Anexo I — Disponibilidad de Código Fuente y Recursos Audiovisuales	73
I.1 Repositorio de Código Fuente y Prototipo Funcional (GitHub)	73
I.2 Registro Audiovisual de Demostración y Pruebas Empíricas (YouTube)	73

Índice de Figuras

Figura 1. Diagrama de Arquitectura de Servicio	27
Figura 2. Diagrama Orquestador-Seguridad	28
Figura 3. Diagrama de Pipeline de ejecución	30
Figura 4. Diagrama de secuencia UML — ciclo de vida de POST /scan	31
Figura 1 (ampliada). Diagrama de Arquitectura de Servicio	69
Figura 2 (ampliada). Diagrama Orquestador-Seguridad	70
Figura 3 (ampliada). Diagrama de Pipeline de ejecución	71
Figura 4 (ampliada). Diagrama de secuencia UML — ciclo de vida de POST /scan	72

Índice de Tablas

Tabla 1. Comparación de plataformas de orquestación y gestión de vulnerabilidades	19
Tabla 2. Componentes del sistema y su función en el pipeline	28
Tabla 3. Resumen de hallazgos — ejecución sobre DVWA del 5 de agosto de 2026	37
Tabla 4. Vulnerabilidades detectadas por OWASP ZAP, por tipo	38
Tabla 5. Rutas descubiertas por fuzzing de directorios (ffuf)	39
Tabla 6. Endpoints confirmados como vulnerables por SQLMap — corrida del 5 de agosto de 2026	39
Tabla 7. Datos extraídos a través de SQL Injection – corrida del 5 de agosto de 2026	39
Tabla 8. Hallazgos clasificados por categoría OWASP Top 10	40
Tabla 9. Cobertura por herramienta en solitario frente al pipeline combinado	42
Tabla 10. Tiempo total del pipeline con y sin caché incremental	44
Tabla 11. Estabilidad de hallazgos entre la corrida sin caché y con caché	45
Tabla 12. Verificación de cumplimiento por objetivo específico	49
Tabla 13. Distribución temporal de la sesión de auditoría manual del capítulo 17	55
Tabla 14. Estructura narrativa, marcas temporales y desglose visual del video demostrativo	74

1. Introducción

1.1 Contexto Global y Panorama de Vulnerabilidades Web

Las aplicaciones web constituyen uno de los pilares fundamentales de la infraestructura digital contemporánea. Organizaciones empresariales, organismos gubernamentales, instituciones académicas y plataformas de comercio electrónico gestionan volúmenes crecientes de información crítica y transaccional a través de sistemas accesibles públicamente mediante internet para interactuar de forma continua con usuarios finales.

Esta centralidad operativa tiene como contrapartida un incremento sistemático en la superficie de ataque y el volumen de incidentes informáticos. De acuerdo con el *2025 Data Breach Investigations Report* (DBIR) de Verizon, los ataques dirigidos a aplicaciones web (*Basic Web Application Attacks*) representaron el 12% del total de brechas de seguridad confirmadas a nivel global durante 2025, experimentando un ascenso frente al 9% registrado en el período precedente, con el agravante de que el 88% de estos incidentes involucró el uso indebido de credenciales comprometidas (Verizon, 2025). En consonancia, el informe *Cost of a Data Breach Report 2025* elaborado por IBM sitúa el costo económico promedio global de una vulneración de datos en 4,44 millones de dólares (IBM, 2025). Estos indicadores cuantitativos evidencian que la debilidad en los perímetros web no constituye un problema periférico, sino un vector de riesgo crítico de elevado impacto patrimonial y reputacional.

Entre las fallas de seguridad recurrentes en los ecosistemas web destacan la inyección de código SQL, las secuencias de comandos en sitios cruzados (Cross-Site Scripting o XSS), las configuraciones de seguridad defectuosas o permisivas, los mecanismos de autenticación y gestión de sesiones débiles, y la exposición involuntaria de rutas o servicios internos. Estas amenazas se encuentran ampliamente categorizadas por la comunidad técnica internacional a través del estándar del proyecto *Open Web Application Security Project* (OWASP Top 10, 2021), que establece el consenso sobre los riesgos más severos a los que se enfrentan las arquitecturas web modernas.

1.2 La Necesidad de Orquestación: Paradigma SOAR y Enfoque DAST Integrado

Frente a este escenario, la evaluación de seguridad tradicional recurre a dos modalidades primarias: auditorías manuales conducidas por analistas de penetración o la utilización aislada de herramientas automatizadas de escaneo dinámico (DAST, *Dynamic Application Security Testing*). El enfoque DAST permite examinar las aplicaciones en su estado de ejecución sin requerir acceso al código fuente, analizando el comportamiento de entradas y respuestas frente a solicitudes anómalas (Qadir et al., 2025). Sin embargo, el empleo independiente y descoordinado de escáneres dinámicos introduce fricciones operativas notables: demanda una intervención humana reiterativa para la parametrización de cada consola, adolece de formatos de salida dispares que dificultan la consolidación de hallazgos y exige una ardua labor de correlación manual para filtrar falsos positivos o reconstruir el cuadro integral de riesgos.

Con el propósito de mitigar esta sobrecarga y dotar de agilidad al diagnóstico, surge en la industria el paradigma SOAR (*Security Orchestration, Automation and Response*), orientado a la orquestación de herramientas heterogéneas, la automatización de flujos repetitivos y la centralización de los reportes de incidentes (Kinyua & Awuah, 2021). No obstante, como se analiza en el Capítulo 7 (Estado del Arte), las plataformas de referencia en este ámbito —tales como Faraday, DefectDojo, TheHive o Shuffle— presentan una marcada orientación hacia la gestión agregada a nivel corporativo o demandan una infraestructura de soporte y consumo de recursos considerablemente pesada (ver Tabla 1). Emerge así una vacancia técnica específica: la necesidad de una solución de orquestación ágil, desacoplada y liviana, que combine el escaneo dinámico general con técnicas de fuzzing activo y explotación dirigida, empaquetada de manera integral bajo contenedores Docker Compose de despliegue directo.

Para abordar dicha brecha, este proyecto implementa un pipeline articulado sobre tres herramientas complementarias de código abierto: OWASP ZAP, que realiza un escaneo dinámico de propósito general (en sus fases pasiva y activa); ffuf, especializado en el descubrimiento de superficie oculta mediante *fuzzing* de alta velocidad sobre directorios y rutas web (Alsaedi et al., 2021); y SQLMap, orientado de manera especializada a la detección profunda y verificación de explotación sobre vulnerabilidades de inyección SQL (Elia et al., 2010), un vector que los escáneres generalistas suelen subestimar o diagnosticar con menor alcance. La combinación estratégica de estas tres capas — descubrimiento de rutas no enlazadas, análisis transversal de vulnerabilidades y verificación exhaustiva de inyecciones— permite maximizar la cobertura global del diagnóstico sin incurrir en la sobrecarga operativa de las plataformas empresariales complejas.

1.3 Propuesta de Valor, Alcance del Prototipo y Entorno Experimental

En este marco metodológico y tecnológico, el presente trabajo final tiene por objetivo diseñar, implementar y validar un orquestador automatizado de herramientas de seguridad web capaz de ejecutar análisis dinámicos secuenciales, normalizar los esquemas sintácticos heterogéneos de cada componente y consolidar un reporte unificado de hallazgos. A partir de dicho núcleo de ejecución por consola, el sistema fue escalado hacia una arquitectura de servicio completa: una API REST asíncrona desarrollada en FastAPI que administra los análisis en segundo plano, persiste el historial de auditorías en base de datos relacional, implementa un módulo de autenticación automatizada para examinar secciones protegidas tras mecanismos de login, proporciona sugerencias de remediación estructuradas mediante inteligencia artificial y expone los resultados a través de un panel web interactivo para el analista.

A efectos de garantizar el rigor metodológico, la reproducibilidad de los ensayos y un marco deontológico estricto, el sistema fue sometido a validación experimental sobre la plataforma vulnerable deliberada *Damn Vulnerable Web Application* (DVWA) desplegada en un entorno de red Docker aislado. Dicho esquema garantiza el pleno cumplimiento de la legislación nacional sobre delitos informáticos (Ley 26.388) y los principios éticos internacionales de la profesión (Código de Ética ACM), cuyas derivaciones jurídicas y operativas se examinan formalmente en el Capítulo 16.

1.4 Estructura del Documento

El presente trabajo se encuentra articulado de la siguiente manera: las secciones 2 a 6 definen la problemática, justificación, metas, interrogantes e hipótesis orientadoras. Las bases teóricas y el estado del arte se examinan en las secciones 7 y 8, dando paso a la delimitación y metodología en los apartados 9 y 10. La arquitectura del sistema, el modelo de servicio y la implementación técnica del pipeline se detallan en los capítulos 11 y 12. La presentación de hallazgos y su debate técnico se abordan en las secciones 13 y 14, condensando las conclusiones y líneas de investigación futuras en el capítulo 15. Por último, la sección 16 aborda las consideraciones éticas y normativas, el capítulo 17 profundiza el desarrollo experimental, el apartado 18 reúne las fuentes bibliográficas y el capítulo 19 consolida los anexos técnicos, incluyendo la disponibilidad del repositorio de código fuente y el registro audiovisual de pruebas empíricas.

2. Planteo del Problema

Pese a la amplia oferta de utilidades especializadas destinadas al diagnóstico de seguridad en plataformas web, la mayoría de las organizaciones tropieza con severas trabas operativas al momento de establecer un esquema de evaluación sistemático, continuo y reproducible. Los vectores de ataque dirigidos contra aplicaciones web representaron el 12% del total de incidentes de seguridad confirmados durante 2025, de los cuales el 88% estuvo vinculado al compromiso de credenciales (Verizon, 2025). Sumado a ello, el costo financiero global de una brecha de datos promedia los 4,44 millones de dólares (IBM, 2025), lo que torna crítico abordar esta problemática desde una perspectiva metodológica e ingenieril sustentada en métricas empíricas.

2.1 Contextualización: La Fricción entre DAST Tradicional y la Agilidad de CI/CD

En el paradigma contemporáneo de ingeniería de software, caracterizado por la adopción generalizada de metodologías ágiles y canalizaciones de integración y despliegue continuos (CI/CD), la frecuencia de publicación de nuevas versiones ha pasado de ciclos mensuales o trimestrales a cadencias diarias o semanales. No obstante, las evaluaciones de seguridad dinámicas (DAST) tradicionales fueron concebidas originalmente para auditorías puntuales, periódicas y de ejecución manual al final del ciclo de vida (Qadir et al., 2025).

Esta divergencia genera un desacople estructural: mientras el equipo de desarrollo demanda respuestas en cuestión de minutos para validar un despliegue, un análisis de seguridad dinámico exhaustivo que opere de manera aislada insume múltiples horas de configuración y ejecución. Como resultado, las auditorías dinámicas son omitidas o relegadas a fases tardías, incrementando el riesgo de promover vulnerabilidades críticas a entornos de producción (SANS Institute, 2024).

2.2 Desarticulación Operativa y Silos de Información

El panorama operativo de las herramientas de código abierto disponibles se encuentra fragmentado. Cada analizador resuelve con alta eficacia un vector específico —el descubrimiento de rutas ocultas mediante fuzzing, el análisis heurístico de vulnerabilidades web o la confirmación de inyecciones en bases de datos—, pero lo hace de forma completamente desacoplada y estanca (Abdulghaffar et al., 2023).

Esta desarticulación se manifiesta principalmente en tres niveles:

1. **Incompatibilidad en los Modelos de Datos de Salida:** OWASP ZAP genera alertas en esquemas XML jerárquicos o JSON complejos con anidamientos profundos; ffuf produce un flujo de objetos JSON línea a línea con métricas de palabras y códigos de estado HTTP; mientras que SQLMap emite volcados interactivos por consola en texto plano y registros de sesión no normalizados. La ausencia de un modelo canónico impide la agregación directa de resultados.
2. **Carencia de Enriquecimiento Dinámico entre Fases:** Los descubrimientos de superficie expuesta realizados por un fuzzer no son comunicados automáticamente al escáner dinámico para guiar su rastreo activo, ni las alertas de parámetros sospechosos son transferidas al motor de explotación especializada para verificar su impacto real.
3. **Ausencia de Memoria Operativa:** Las ejecuciones tradicionales carecen de mecanismos de persistencia que almacenen el estado de corridas previas. Cada nueva auditoría evalúa desde cero todos los endpoints, consumiendo ancho de banda y tiempo de procesamiento de forma redundante sobre componentes que no han sufrido modificaciones en su código fuente.

2.3 Taxonomía Estructurada de Problemas Identificados

A partir del análisis operativo y de la literatura especializada en automatización de seguridad (Kinyua & Awuah, 2021; Zhang et al., 2023), se identifican seis problemas críticos que obstaculizan la efectividad de las pruebas DAST en entornos productivos:

1. **Silos de información e ineficiencia en la correlación cruzada:** Las herramientas operan en entornos estancos y producen reportes disímiles, imposibilitando la correlación automatizada de vectores e impidiendo que los hallazgos de superficie alimenten dinámicamente el análisis activo.
2. **Sobrecarga operativa por tareas manuales repetitivas:** Los profesionales de seguridad deben intervenir manualmente para lanzar cada utilidad de forma secuencial, configurar credenciales de sesión en consolas heterogéneas y consolidar los resultados artesanalmente, consumiendo tiempo valioso en labores administrativas de bajo impacto técnico.
3. **Fricción temporal e incompatibilidad con la cadencia de CI/CD:** La ejecución secuencial no coordinada demanda lapsos incompatibles con los despliegues ágiles. La ausencia de mecanismos de caché incremental agrava este escenario, forzando escaneos completos redundantes ante cambios mínimos.
4. **Deficiencias en la reproducibilidad del entorno de ejecución:** La variabilidad en las dependencias del sistema operativo, las versiones de los intérpretes y las configuraciones locales dificulta contrastar resultados obtenidos en distintos momentos, obstaculizando la validación fiable de los procesos de remediación.
5. **Brecha en la comunicación ejecutiva y dispersión de métricas:** Los reportes nativos de cada herramienta priorizan detalles técnicos atomizados sin proporcionar resúmenes consolidados de nivel directivo ni clasificaciones de severidad global que faciliten la toma de decisiones basada en riesgo.
6. **Fatiga de alertas y saturación cognitiva del equipo auditor:** La dispersión de logs no normalizados genera un volumen masivo de avisos secundarios y posibles falsos positivos, elevando la probabilidad de que vulnerabilidades de severidad crítica pasen inadvertidas por sobrecarga informativa (Zhang et al., 2023).

2.4 Brecha Operativa a Resolver por el Orquestador

Las plataformas comerciales de orquestación y gestión de vulnerabilidades que abordan estos problemas suelen exigir despliegues de infraestructura pesados, configuraciones complejas y modelos de suscripción de elevado costo financiero (DefectDojo, 2025; Faraday Security, 2025), lo que las torna inaccesibles para organizaciones medianas o proyectos con recursos acotados.

Surge así una brecha operacional concreta: la necesidad de una solución de software liviana, estructurada bajo estándares abiertos y desplegable en contenedores aislados, que automatice la coordinación secuencial de herramientas DAST complementarias. Dicho sistema debe normalizar los hallazgos en un esquema unificado, habilitar el enriquecimiento cruzado entre etapas, optimizar las ventanas de ejecución mediante persistencia y caché incremental, y ofrecer visibilidad tanto técnica como ejecutiva, sentando las bases para los objetivos formulados en el Capítulo 4.

3. Justificación

La fundamentación de esta investigación se estructura sobre cuatro dimensiones complementarias que articulan las competencias formativas de la carrera, las demandas técnicas de la ingeniería de software moderna, la viabilidad económica y la transferencia hacia el medio socio-productivo.

3.1 Justificación Académica e Integración Formativa en la TUP

Desde la perspectiva académica e institucional, el desarrollo del presente Trabajo Final de Grado materializa la convergencia interdisciplinaria de las principales áreas de conocimiento que configuran el plan de estudios de la Tecnicatura Universitaria en Programación (TUP) de la Universidad Tecnológica Nacional — Facultad Regional Mendoza. Lejos de constituir una aplicación aislada de conceptos, el orquestador requirió articular de manera transversal:

1. **Programación Orientada a Objetos y Buenas Prácticas:** Implementación robusta en Python 3.11, aplicando principios de diseño orientado a objetos, modularización, tipado estático auxiliar y gestión estructurada de excepciones en la capa del orquestador central (Python Software Foundation, 2024).
2. **Diseño y Arquitectura de Software:** Adopción rigurosa del patrón de diseño Pipeline y de los principios SOLID —en particular, el principio de responsabilidad única y el de abierto/cerrado— para desacoplar el ciclo de vida de la ejecución respecto de los módulos de parsing y persistencia (Buschmann et al., 1996; Martin, 2003).
3. **Bases de Datos y Persistencia Estructurada:** Modelado y gestión de persistencia de sesiones, metadatos, historial de escaneos y caché incremental mediante bases de datos relacionales con soporte SQLite y proyección hacia PostgreSQL, asegurando integridad transaccional.
4. **Redes, Protocolos y Comunicaciones:** Manipulación analítica del protocolo HTTP/HTTPS, inspección de cabeceras, gestión de códigos de respuesta, manejo de cookies de sesión y autenticación contextual requerida para la evaluación profunda de endpoints.
5. **Infraestructura y Contenedores:** Aislamiento de dependencias y entornos de ejecución heterogéneos mediante la tecnología de contenedores Docker y orquestación liviana con Docker Compose (Docker Inc., 2024).
6. **Servicios Web e Interfaz de Usuario:** Extensión del núcleo analítico hacia una arquitectura de servicios orientada a la web, implementando una API REST asíncrona mediante FastAPI y una interfaz gráfica interactiva en React.

Asimismo, el proyecto aporta un caso de estudio reproducible para la enseñanza de la ciberseguridad ofensiva y las pruebas de software en el ámbito universitario, permitiendo a los estudiantes contrastar empíricamente cómo interactúan los vectores de vulnerabilidad en entornos de laboratorio controlados.

3.2 Justificación Técnica: Paradigma Shift-Left y Enriquecimiento Cruzado

Desde la perspectiva técnica y metodológica, la integración continua y la entrega continua (CI/CD) han acelerado radicalmente el ciclo de vida del desarrollo de software. No obstante, las evaluaciones de seguridad dinámicas (DAST) tradicionales suelen introducir una severa fricción operativa: requieren configuraciones manuales complejas, generan salidas incompatibles y demandan lapsos temporales incompatibles con la agilidad requerida en los despliegues modernos.

El orquestador desarrollado resuelve esta problemática mediante dos contribuciones arquitectónicas clave:

1. **Adopción del Paradigma Shift-Left:** Permite trasladar el análisis de seguridad dinámico hacia las etapas tempranas del desarrollo. Al condensar el escaneo en una rutina automatizada que insume entre 4 y 14 minutos —según la activación o no de la caché incremental documentada en la Tabla 10—, los equipos de desarrollo pueden ejecutar auditorías periódicas sin obstaculizar el flujo de entregas.
2. **Enriquecimiento Cruzado de Herramientas (Cross-Tool Feeding):** A diferencia de la ejecución en silos aislados, el pipeline implementa un flujo coordinado donde los hallazgos de una fase nutren la ejecución de la siguiente. El fuzzer de alto rendimiento (ffuf) descubre directorios y rutas ocultas que escapan al rastreo estándar; dicha superficie de ataque descubierta es transferida de forma inmediata al escáner dinámico (OWASP ZAP) para su evaluación activa, y aquellos endpoints sospechosos de procesar parámetros dinámicos vulnerables son analizados en profundidad por el motor especializado en inyecciones (SQLMap).

Este enfoque reduce drásticamente la tasa de omisión derivada de herramientas desarticuladas y minimiza el error humano intrínseco a la configuración manual de múltiples utilidades de consola.

3.3 Justificación Económica y Modelo Cuantitativo de Retorno de Inversión (ROI)

Desde el punto de vista económico, las evaluaciones de seguridad tradicionales representan una barrera financiera significativa. Una auditoría de seguridad web externa o prueba de penetración manual profesional requiere habitualmente entre 20 y 40 horas de labor técnica cualificada, con honorarios que oscilan entre 50 y 150 USD por hora en el mercado especializado (SANS Institute, 2024).

Bajo un escenario estándar representativo de 30 horas profesionales a una tarifa media conservadora de 100 USD por hora:

$$C_{\text{auditoría}} = 30 \text{ horas} \times 100 \frac{\text{USD}}{\text{hora}} = 3.000 \text{ USD}$$

Para una organización que implemente un esquema básico de revisiones trimestrales, el costo operativo anual directo asciende a:

$$C_{\text{anual}} = 3.000 \text{ USD} \times 4 \text{ revisiones} = 12.000 \text{ USD/año}$$

Este volumen de erogación resulta económicamente inviable para pequeñas y medianas empresas (PyMEs), centros académicos o proyectos emergentes, induciendo a prescindir de controles de seguridad dinámicos antes del paso a producción. Por otra parte, las plataformas comerciales de orquestación y gestión de vulnerabilidades reservan sus capacidades avanzadas de automatización para suscripciones corporativas de costo prohibitivo (DefectDojo, 2025; Faraday Security, 2025).

Frente a este escenario, la solución implementada demuestra un balance de retorno de inversión (ROI) altamente favorable:

- **Costo Directo de Licenciamiento:** 0 USD, al basarse íntegramente en tecnologías de código abierto de grado industrial (OWASP ZAP, ffuf, SQLMap, Docker, FastAPI, React).

- **Optimización Temporal del Recurso Humano:** El pipeline automatizado ejecuta las tareas rutinarias de reconocimiento, fuzzing masivo y escaneo activo en un intervalo acotado de 4 a 14 minutos, permitiendo que el profesional concentre su tiempo exclusivamente en el triaje y validación final de hallazgos críticos.
- **Mitigación Preventiva de Costos por Incidentes:** Según el informe global de IBM (2025), el costo promedio de una filtración de datos asciende a 4,44 millones de dólares. Detectar fallas críticas como inyecciones SQL o credenciales expuestas en fases tempranas previene impactos financieros y legales de orden catastrófico para la organización.

3.4 Transferencia Tecnológica y Replicabilidad en PyMEs y Ámbito Educativo

Finalmente, el proyecto posee una clara vocación de transferencia tecnológica. La arquitectura no se restringe a un script de uso personal, sino que se formaliza como un prototipo desacoplado, modular y reproducible:

1. **Democratización para el Ecosistema PyME:** Brinda a organizaciones sin departamentos formales de ciberseguridad una herramienta accesible, desplegable en minutos mediante un único comando de orquestación (`docker compose up`), capaz de entregar un reporte consolidado en JSON y Markdown sin costos recurrentes.
2. **Valor Pedagógico y Transferencia Académica:** Funciona como un entorno de laboratorio para que cátedras de desarrollo de software y seguridad puedan experimentar con la automatización DAST y el paradigma Shift-Left sobre blancos controlados como DVWA.
3. **Extensibilidad Arquitectónica:** La separación limpia entre la capa de ejecución, la API REST asíncrona y la interfaz web asegura que otros equipos puedan extender el sistema integrando nuevas herramientas de escaneo (ej. análisis de contenedores o escaneo SSL) mediante el patrón de diseño establecido.

4. Objetivos

4.1 Objetivo General

Diseñar e implementar un sistema automatizado que permita orquestar múltiples herramientas de análisis de seguridad web para detectar vulnerabilidades en aplicaciones web, consolidar los resultados en reportes unificados y exponer el sistema como un servicio reutilizable.

El propósito central persigue resolver la fragmentación operativa y el elevado costo temporal que impone la ejecución artesanal de auditorías dinámicas de seguridad web. Al integrar herramientas heterogéneas bajo un flujo coordinado y desatendido, la investigación busca desplazar los análisis hacia etapas tempranas del ciclo de vida del software (paradigma *Shift-Left*), dotando a los equipos de desarrollo y aseguramiento de la calidad de un artefacto de software extensible y reproducible que unifique el descubrimiento de superficie, la explotación controlada y el reporte estructurado de hallazgos.

4.2 Objetivos Específicos

Para alcanzar el objetivo general, la investigación tecnológica aplicada se descompone en nueve objetivos específicos que abarcan el ciclo integral de ingeniería y validación experimental. Estos objetivos se articulan en torno a tres ejes metodológicos y funcionales complementarios:

- **Eje de Integración y Detección Dinámica (OE1, OE3, OE6):** orientado a coordinar la triada de escaneo dinámico (OWASP ZAP, ffuf y SQLMap) mediante mecanismos de enriquecimiento cruzado, parseo canónico de salidas heterogéneas y autenticación programática de sesiones, operando directamente sobre las variables dependientes de cobertura de análisis y número de vulnerabilidades detectadas (sección 10.3).
- **Eje de Arquitectura de Software, Contenerización y Servicios (OE2, OE4, OE7):** enfocado en concebir una estructura modular desacoplada basada en principios SOLID, aislar y aprovisionar los componentes en contenedores Docker Compose sobre una red dedicada, y exponer el pipeline como una API REST asíncrona con persistencia de historial, garantizando la manipulación controlada de la variable independiente de automatización y el rigor de las variables de control (sección 10.3).
- **Eje de Validación Experimental, Comunicación y Usabilidad (OE5, OE8, OE9):** destinado a someter el artefacto a evaluación empírica en un entorno de laboratorio controlado (DVWA) bajo los criterios de validación formales de la sección 10.6, sintetizar los resultados en reportes diferenciados con estimación de riesgo, y proveer una interfaz web interactiva que democratice la operación del sistema para usuarios no especialistas.

A continuación se detallan los nueve objetivos específicos formulados para el proyecto:

1. Integrar herramientas de análisis de seguridad web (OWASP ZAP, ffuf y SQLMap) en un flujo de ejecución automatizado, enriqueciendo el análisis de cada herramienta con los hallazgos de las demás.
2. Diseñar una arquitectura modular que permita extender el sistema fácilmente mediante la incorporación de nuevas herramientas o módulos de análisis.
3. Implementar parsers especializados que normalicen los resultados generados por cada herramienta en un formato homogéneo.
4. Desplegar el sistema en un entorno contenerizado utilizando Docker, garantizando la reproducibilidad del entorno de ejecución.
5. Evaluar el funcionamiento del sistema en un entorno de pruebas controlado (DVWA), verificando el cumplimiento de los criterios de validación definidos en la sección 10.6 (funcionalidad, reproducibilidad, cobertura mínima de tres categorías OWASP Top 10 y consolidación completa de hallazgos).
6. Implementar un mecanismo de autenticación automática que permita al orquestador acceder a secciones protegidas de la aplicación objetivo durante el escaneo.
7. Exponer el orquestador como un servicio mediante una API REST que ejecute los análisis de forma asíncrona, persista el historial de escaneos y permita su consulta posterior.
8. Generar automáticamente dos variantes de reporte —técnico y ejecutivo— con clasificación de riesgo global, orientadas a audiencias técnicas y no técnicas respectivamente.
9. Desarrollar una interfaz web que consuma la API del orquestador y permita lanzar escaneos, visualizar su progreso y consultar los hallazgos sin requerir acceso a la línea de comandos.

Nota de verificación metodológica: El grado de cumplimiento, la trazabilidad documental y la contrastación empírica de cada uno de estos nueve objetivos específicos se auditan de forma exhaustiva y positiva en la **Tabla 12** de la subsección 15.1.1.

5. Preguntas de Investigación

En concordancia con el paradigma de investigación tecnológica aplicada de tipo constructivo adoptado (sección 10.1), las preguntas de investigación actúan como directrices epistémicas que delimitan los problemas técnicos, metodológicos y operativos abordados durante la creación y validación del artefacto de software. Cada interrogante examina una tensión crítica del ciclo DevSecOps: la fricción temporal en la integración continua, el aislamiento de las herramientas especializadas, la heterogeneidad de esquemas de datos, la complejidad de reproducción del entorno y la transición desde utilitarios de consola hacia plataformas de servicio.

A continuación se presentan las cinco preguntas de investigación que guían el desarrollo y la evaluación del proyecto, explicitando para cada una su fundamentación teórica y su articulación metodológica:

1. **(PI1)** ¿En qué medida un sistema de orquestación desarrollado en Python mejora, en términos cualitativos y de tiempo de re-ejecución, la coordinación de múltiples herramientas de seguridad web frente a su ejecución manual e independiente?
 - **Fundamentación y brecha conceptual:** Aborda la fricción operativa y los cuellos de botella generados por la ejecución interactiva, secuencial y fragmentada de pruebas de seguridad por parte de auditores humanos (Planteo del Problema, Capítulo 2). Indaga si la coordinación desatendida del flujo y la incorporación de mecanismos de caché incremental permiten mitigar sustancialmente los tiempos de respuesta sin degradar la exhaustividad ni la consistencia de los hallazgos.
 - **Articulación metodológica:** Vinculada operativamente con los objetivos específicos OE1 y OE5, y con las hipótesis H1 (reducción del esfuerzo manual) y H4 (optimización temporal por caché). Su respuesta empírica se desarrolla en la sección 15.1, sustentada por las mediciones de las Tablas 10 y 11 (reducción temporal del 68,9% por caché) y la contrastación del esfuerzo manual documentada en la sección 14.2 y la Tabla 13 (Capítulo 17).
2. **(PI2)** ¿La integración de herramientas open source en un pipeline unificado —donde los hallazgos de una herramienta enriquecen el análisis de otra— permite ampliar la cobertura de detección respecto al uso individual de cada herramienta?
 - **Fundamentación y brecha conceptual:** Examina el problema de los silos de información característico de las herramientas DAST tradicionales, las cuales operan de forma estanca y sin retroalimentación contextual. Cuestiona si el encadenamiento secuencial con inyección cruzada (*cross-tool feeding*: transferencia de rutas descubiertas por ffuf al árbol de ZAP y selección heurística de URLs candidatas hacia SQLMap) logra descubrir vectores que permanecen invisibles ante el análisis individual y aislado de cada motor.
 - **Articulación metodológica:** Vinculada con los objetivos específicos OE1 y OE5, y con la hipótesis H2 (cobertura multidimensional por enriquecimiento cruzado). Su contrastación empírica se documenta en la sección 14.1 y en la respuesta de la sección 15.1, respaldada por la Tabla 8 y la Tabla 9, donde el pipeline unificado cubrió tres categorías del OWASP Top 10 frente a una única categoría por herramienta en solitario.

3. **(PI3)** ¿Es posible generar un formato unificado de resultados a partir de herramientas que producen salidas en formatos heterogéneos?
 - **Fundamentación y brecha conceptual:** Responde a la fragmentación sintáctica y semántica derivada de coordinar motores de diversa índole (árbol jerárquico XML/JSON de ZAP, flujo continuo en JSON de ffuf y registros de texto plano de SQLMap). Indaga la factibilidad de transformar dicha diversidad hacia un modelo de datos homogéneo y normalizado, capaz de preservar los atributos esenciales de cada vulnerabilidad y posibilitar su ingesta automática por herramientas de gestión o interfaces web.
 - **Articulación metodológica:** Vinculada con los objetivos específicos OE3 y OE8, y con la hipótesis H3 (normalización estructurada). Se fundamenta en la arquitectura de parsers desacoplados (sección 12.7) y en la consolidación del esquema canónico *resultado_unificado.json* (Anexo D), validada positivamente en la sección 15.1.
4. **(PI4)** ¿Qué ventajas ofrece una arquitectura modular y contenerizada para sistemas de análisis de seguridad automatizados en términos de extensibilidad y reproducibilidad?
 - **Fundamentación y brecha conceptual:** Indaga en el dilema de la portabilidad y la resolución de dependencias antagónicas propio del software de ciberseguridad. Examina en qué medida la adhesión a patrones de diseño desacoplados (SOLID) y la infraestructura inmutable basada en contenedores Docker mitigan el riesgo de regresiones y garantizan la reproducibilidad exacta del banco de pruebas, permitiendo incorporar nuevos módulos de escaneo sin alterar el núcleo del sistema.
 - **Articulación metodológica:** Vinculada con los objetivos específicos OE2 y OE4. Su resolución se sustenta en la arquitectura de paquetes (sección 11.2), el despliegue multicontenedor sobre la red *sec-net* (sección 12.1) y la verificación del criterio de reproducibilidad (100% en hallazgos de ZAP y SQLMap, Tabla 11), ratificada en la sección 15.1.
5. **(PI5)** ¿Puede un sistema de este tipo extenderse, sin modificar su lógica de escaneo central, hacia un servicio persistente con interfaz web y capacidades de seguimiento histórico de vulnerabilidades?
 - **Fundamentación y brecha conceptual:** Explora la superación de la brecha entre utilitarios de línea de comandos de uso exclusivo para especialistas y plataformas centralizadas integrables en la cultura DevSecOps. Evalúa si el desacoplamiento arquitectónico del orquestador permite añadir asincronía, persistencia relacional híbrida (SQLite/PostgreSQL) y una interfaz de usuario intuitiva sin requerir modificaciones sobre el flujo de escaneo subyacente.
 - **Articulación metodológica:** Vinculada con los objetivos específicos OE6, OE7 y OE9. Se sustenta en el desarrollo de la Fase 7 (sección 10.2), la API REST en FastAPI, la persistencia en base de datos y la SPA en React (sección 12.10, Anexos F y G), corroborada en las conclusiones de la sección 15.1.

6. Hipótesis

En concordancia con el diseño de investigación tecnológica aplicada y el enfoque cuantitativo adoptado (sección 10.1), las hipótesis de trabajo postulan relaciones funcionales entre las decisiones arquitectónicas del orquestador y la optimización de los procesos de auditoría web. Cada hipótesis explicita su formulación, sus variables y su criterio de contrastación empírica:

H1 — Reducción del Esfuerzo Manual mediante Automatización: La automatización del proceso de análisis de seguridad web mediante un pipeline de orquestación desacoplado reduce el esfuerzo manual necesario para detectar y correlacionar vulnerabilidades, al sustituir la ejecución interactiva independiente de cada herramienta y la consolidación artesanal de reportes por un flujo desatendido y centralizado.

- **Variables:** *Variable Independiente (VI):* Modalidad de ejecución (pipeline en Python vs. consola interactiva manual). *Variable Dependiente (VD):* Esfuerzo operativo (tiempo de intervención y acciones de correlación).
- **Contrastación empírica:** Contrastada en la sección 14.2 y respaldada en el Capítulo 17 (Tabla 13: 125 min totales = 75 min desatendidos vs. 50 min manuales).

H2 — Cobertura Multidimensional por Enriquecimiento Cruzado: La integración secuencial de múltiples herramientas dinámicas (fuzzing con ffuf, escaneo activo con OWASP ZAP y explotación con SQLMap), donde los hallazgos de una fase enriquecen dinámicamente la superficie de las subsecuentes, permite alcanzar una cobertura de detección mayor que el uso individual de cada herramienta.

- **Variables:** *VI:* Flujo integrado con retroalimentación (*cross-tool feeding*: ffuf → ZAP → SQLMap). *VD:* Cobertura de detección (endpoints analizados y categorías OWASP Top 10 identificadas).
- **Contrastación empírica:** Contrastada en la sección 14.1 y ratificada en las Tablas 8 y 9 (cobertura combinada de 3 categorías OWASP y 34 endpoints unificados frente a coberturas individuales parciales).

H3 — Normalización Estructurada e Interoperabilidad de Salidas Heterogéneas: La incorporación de una capa de parseo desacoplada y especializada por herramienta transforma las salidas sintácticas y semánticas heterogéneas (XML/JSON de ZAP, streaming de ffuf y texto de SQLMap) en un esquema canónico homogéneo, facilitando la interpretación integral de los hallazgos y su interoperabilidad con servicios web.

- **Variables:** *VI:* Arquitectura de parsers independientes basados en interfaces modulares. *VD:* Uniformidad estructural e interpretabilidad (esquema canónico consumible por API REST y frontend).
- **Contrastación empírica:** Contrastada en la sección 12.7 y corroborada en la respuesta a PI3 (sección 15.1), respaldada por el esquema canónico `resultado_unificado.json` (Anexo D).

H4 — Optimización Temporal Mediante Caché Incremental: Un mecanismo de caché incremental respaldado en base de datos relacional reduce significativamente el tiempo de procesamiento en ejecuciones repetidas del pipeline sobre un mismo objetivo, al omitir pruebas redundantes sobre componentes previamente analizados sin hallazgos, preservando intactas la exhaustividad y reproducibilidad.

- **Variables:** *VI:* Caché incremental persistente con identificación de estado por hash. *VD:* Tiempo de ejecución en corridas sucesivas y consistencia de hallazgos sobre el mismo objetivo.
- **Contrastación empírica:** Contrastada en la sección 14.3 y en la respuesta a PI1 (sección 15.1), respaldada por las Tablas 10 y 11 (reducción del 68,9% del tiempo: de 14m 06s a 4m 23s, con 100% de consistencia en 48 alertas ZAP y 8 inyecciones SQLMap).

7. Estado del Arte

En los últimos años se han desarrollado numerosas herramientas destinadas al análisis de seguridad en aplicaciones web, clasificables en distintas categorías según su enfoque y funcionalidad.

Los escáneres de vulnerabilidades como OWASP ZAP o Burp Suite realizan análisis dinámico (DAST) mediante técnicas de escaneo activo y pasivo. Shahid et al. (2022), en un estudio comparativo de parámetros de seguridad en aplicaciones web publicado en Applied Sciences, analizan las tendencias actuales y las direcciones futuras de las herramientas de este tipo, señalando la fragmentación de enfoques entre escáneres de propósito general y herramientas especializadas como una de las limitaciones actuales del ecosistema (Shahid et al., 2022). En la misma línea, Qadir et al. (2025) evaluaron empíricamente nueve herramientas SAST y DAST de código abierto sobre setenta y cinco aplicaciones web reales, y encontraron que ninguna herramienta individual —estática o dinámica— cubre por sí sola la totalidad de las categorías del OWASP Top 10, lo que refuerza la necesidad de combinarlas (Qadir et al., 2025).

Althunayyan et al. (2022) evaluaron además cinco escáneres de caja negra frente a una aplicación de comercio electrónico moderna y encontraron diferencias sustanciales en la tasa de detección de vulnerabilidades de inyección entre herramientas, lo que respalda empíricamente la decisión de este proyecto de no depender de un único escáner (Althunayyan et al., 2022). En una línea directamente relevante para el diseño de este proyecto, Abdulghaffar, Elmrahit y Yousefi (2023) evaluaron un enfoque de pruebas de penetración automatizadas que combina múltiples escáneres de vulnerabilidades sobre un mismo objetivo y muestran que la combinación de herramientas heterogéneas amplía la cobertura de detección respecto del uso de un escáner aislado, un hallazgo que sustenta directamente la decisión arquitectónica de este trabajo de orquestar ZAP, ffuf y SQLMap en lugar de depender de una única herramienta (Abdulghaffar et al., 2023).

Las herramientas de fuzzing como ffuf o dirsearch permiten descubrir rutas ocultas o endpoints expuestos mediante el envío sistemático de requests generados a partir de wordlists, complementando el descubrimiento de superficie de ataque que realiza el spider de un escáner DAST. Alsaedi et al. (2021) relevan en su encuesta sobre fuzzing de caja negra el estado de las técnicas de construcción de ataques para aplicaciones web, y clasifican a las herramientas de descubrimiento de rutas como una etapa complementaria —no sustituta— del escaneo activo (Alsaedi et al., 2021). Zhang et al. (2023) proponen además un enfoque de escaneo sensible al estado de la aplicación que reduce el tiempo desperdiciado en visitar páginas equivalentes durante el fuzzing, un problema de eficiencia análogo al que este proyecto aborda mediante el mecanismo de caché incremental descrito en la sección 11.1 (Zhang et al., 2023).

Por otra parte, herramientas especializadas como SQLMap se enfocan en la detección y explotación de un vector específico —inyección SQL— con mayor profundidad que la que ofrecen los escáneres generales para esa misma categoría de vulnerabilidad. Elia et al. (2010), en un estudio experimental comparativo de herramientas de detección de inyección SQL mediante inyección de ataques, encontraron que la efectividad de estas herramientas varía sustancialmente según el mecanismo de detección utilizado y advierten sobre los falsos negativos de los enfoques basados en coincidencia de texto —una observación directamente relevante para la limitación del parser de SQLMap de este proyecto, discutida en la sección 12.7 (Elia et al., 2010).

Más allá de estas herramientas puntuales, existen plataformas de código abierto orientadas específicamente a la orquestación y gestión de resultados de seguridad, generalmente agrupadas bajo el concepto de SOAR (Security Orchestration, Automation and Response).

Kinyua y Awuah (2021) caracterizan a las plataformas SOAR como la integración de herramientas de seguridad dispares, la automatización de tareas repetitivas de análisis y la estandarización de la respuesta ante hallazgos, y señalan como dirección de investigación futura la incorporación de

componentes de aprendizaje automático para priorizar alertas (Kinyua & Awuah, 2021): Faraday centraliza los resultados de múltiples escáneres en un repositorio colaborativo; DefectDojo gestiona el ciclo de vida de vulnerabilidades con seguimiento histórico y métricas por proyecto; TheHive y Shuffle automatizan flujos de respuesta ante incidentes mediante playbooks configurables, de forma conceptualmente similar a como este proyecto usa n8n para disparar el análisis de sugerencias de mitigación. Estas plataformas resuelven el problema de integración y correlación de resultados que motiva este trabajo, pero requieren instalación y configuración considerablemente más complejas, y varias ofrecen sus funcionalidades avanzadas (gestión de equipos, integraciones empresariales) bajo modelos de licenciamiento comercial.

Tabla 1. Comparación de plataformas de orquestación y gestión de vulnerabilidades

Plataforma	Alcance funcional	Curva de instalación	Costo	Automatización del pipeline
Faraday	Centraliza resultados de múltiples escáneres en un repositorio colaborativo	Servidor dedicado y configuración de agentes/plugins	Community gratuito / Enterprise comercial	Centraliza resultados; no ejecuta ni coordina el escaneo en sí
DefectDojo	Gestión del ciclo de vida de vulnerabilidades, métricas por proyecto	Despliegue vía Docker Compose; requiere configurar importadores por herramienta	Community gratuito / Pro comercial	Importa resultados ya generados; no orquesta la ejecución de las herramientas
TheHive / Shuffle	Automatización de respuesta a incidentes mediante playbooks configurables	Infraestructura compuesta (TheHive + Cortex + integraciones)	Community gratuito / soporte comercial	Alto en respuesta a incidentes; no está orientado a fuzzing ni escaneo web
Orquestador de Seguridad (este proyecto)	Orquestación end-to-end de ZAP, ffuf y SQLMap con enriquecimiento cruzado entre herramientas	Un comando (docker compose up)	Gratuito, sin licenciamiento por niveles	Alto: coordina, enriquece y consolida automáticamente todo el pipeline de escaneo

Nota. Elaboración propia a partir de la documentación oficial de cada plataforma (Faraday Security, 2025; DefectDojo, 2025; TheHive Project, 2025; Shuffle, 2025) y de la descripción funcional del sistema desarrollado.

Frente a este panorama, el proyecto presentado en este trabajo propone una alternativa de menor escala: un orquestador propio en Python que integra tres herramientas puntuales, con persistencia de historial, API y un panel web simplificado, priorizando la simplicidad de despliegue y la reproducibilidad sobre la amplitud funcional de una plataforma SOAR completa.

8. Marco Teórico y Conceptual

8.1 Fuzzing

El fuzzing es una técnica de pruebas de seguridad que consiste en enviar entradas inesperadas o malformadas a un sistema con el objetivo de identificar comportamientos anómalos o vulnerabilidades (Alsaedi et al., 2021). Se distinguen tres enfoques según el nivel de conocimiento del sistema bajo prueba: black-box (sin conocimiento interno, basado únicamente en la interfaz expuesta), grey-box (con conocimiento parcial, por ejemplo instrumentación de cobertura) y white-box (con acceso completo al código fuente). La herramienta ffuf utilizada en este proyecto corresponde a un fuzzing black-box de tipo directory fuzzing: un subtipo específico de fuzzing web orientado al descubrimiento de rutas, distinto del fuzzing de parámetros o del fuzzing de protocolos de red y binarios (Alsaedi et al., 2021).

8.2 Escaneo de Vulnerabilidades

Se distingue entre DAST (Dynamic Application Security Testing), que analiza la aplicación en ejecución sin acceso a su código fuente —el enfoque utilizado por OWASP ZAP en este proyecto—, y SAST (Static Application Security Testing), que analiza el código fuente sin ejecutarlo. Qadir et al. (2025) comparan empíricamente ambos enfoques sobre setenta y cinco aplicaciones reales y concluyen que son complementarios antes que sustitutos: las herramientas SAST detectan mejor vulnerabilidades de severidad alta en etapas tempranas, mientras que las DAST —como las utilizadas en este proyecto— identifican fallas que solo se manifiestan en tiempo de ejecución (Qadir et al., 2025). Dentro del enfoque DAST se distinguen además el escaneo pasivo (análisis del tráfico sin modificar solicitudes) y el escaneo activo (envío de payloads para verificar vulnerabilidades).

8.3 Contenedores Docker

Docker permite ejecutar aplicaciones dentro de contenedores aislados que comparten el kernel del sistema operativo anfitrión, a diferencia de la virtualización completa, que emula hardware y ejecuta un sistema operativo independiente por cada máquina virtual. Para este proyecto, Docker se eligió por su menor sobrecarga de recursos y su tiempo de arranque casi instantáneo, adecuado para levantar y destruir entornos de laboratorio repetidamente. Esta elección tiene una implicancia de seguridad relevante: al compartir kernel, un contenedor mal configurado representa una superficie de escape potencial hacia el host. Javed y Toor (2021), en una evaluación de herramientas de detección de vulnerabilidades en contenedores (Clair, Anchore y Microscanner) sobre imágenes públicas de Docker Hub, encontraron que ninguna de las tres herramientas detecta de forma confiable vulnerabilidades a nivel de paquetes de aplicación —solo del sistema operativo base—, lo que refuerza por qué este proyecto no delega el análisis de seguridad de la aplicación al propio motor de contenedores, sino a herramientas dedicadas de DAST y fuzzing que operan sobre el servicio en ejecución (Javed & Toor, 2021). Por este motivo, los servicios de este proyecto se ejecutan sin privilegios extendidos.

8.4 OWASP Top 10

El proyecto OWASP publica periódicamente una lista de las diez categorías de riesgo más críticas para aplicaciones web (OWASP Foundation, 2021). Además de la inyección (A03:2021) y las fallas de configuración (A05:2021), el sistema desarrollado también detecta hallazgos de la categoría A01:2021 (Broken Access Control, mediante rutas administrativas descubiertas por ffuf) y, a través de sus componentes de autenticación, resulta relevante para A07:2021 (fallas de identificación y autenticación).

8.5 Pipeline de Datos y Arquitectura Modular

El pipeline implementado sigue el patrón arquitectónico conocido como Pipeline Pattern (o Pipes and Filters), en el que los datos fluyen a través de una secuencia de etapas de transformación, cada una con una responsabilidad única; este patrón fue formalizado por Buschmann, Meunier, Rohnert,

Sommerlad y Stal (1996) en su catálogo de patrones de arquitectura de software como una solución recurrente para sistemas que procesan datos en etapas independientes y reemplazables, propiedad que en este proyecto se aprovecha para intercalar la inyección de rutas de ffuf en la sesión de ZAP sin modificar el resto de las etapas del pipeline (Buschmann et al., 1996). La separación entre runners, scanners, parsers y workflow responde adicionalmente al principio de responsabilidad única (Single Responsibility Principle), uno de los cinco principios SOLID de diseño orientado a objetos sistematizados por Martin (2003), según el cual un módulo debe tener una única razón para cambiar; en este proyecto, esa separación es lo que permite sustituir o extender un scanner o un parser sin afectar al resto del pipeline (Martin, 2003).

8.6 Orquestación y Automatización de Seguridad (SOAR)

El concepto de SOAR (Security Orchestration, Automation and Response) engloba las tecnologías que permiten integrar herramientas de seguridad dispares, automatizar tareas repetitivas de análisis y estandarizar la respuesta ante hallazgos (Kinyua & Awuah, 2021). La extensión de este proyecto hacia una API con ejecución asíncrona, persistencia de historial y notificación mediante webhooks a un flujo de n8n lo posiciona conceptualmente dentro de esta categoría, aunque a una escala considerablemente más acotada que las plataformas SOAR comerciales o de código abierto de mayor madurez descritas en el Estado del Arte. Kinyua y Awuah (2021) identifican la incorporación de aprendizaje automático para la priorización de alertas como una de las principales direcciones futuras del campo; este proyecto no implementa dicha capacidad, pero sí incorpora un componente de sugerencias de mitigación asistidas por IA (sección 12.10), conceptualmente cercano a esa dirección de trabajo.

9. Alcances y Limitaciones

9.1 Alcances

El presente trabajo final de graduación abarca el diseño, la implementación, el despliegue y la validación experimental de un sistema de orquestación de seguridad informática concebido para coordinar herramientas dinámicas heterogéneas en entornos controlados de laboratorio. A partir de una arquitectura desacoplada y orientada a servicios, el alcance funcional del artefacto desarrollado comprende tres bloques operativos plenamente verificados:

a) Núcleo de Detección Dinámica y Enriquecimiento Cruzado:

- Coordinación automatizada y secuencial de la triada DAST de código abierto: OWASP ZAP para escaneo pasivo y activo de vulnerabilidades generales, ffuf para el descubrimiento de superficie oculta mediante *fuzzing* de directorios con diccionarios parametrizables, y SQLMap para la detección profunda y verificación de inyecciones SQL.
- Mecanismo de enriquecimiento cruzado inter-herramienta: realimentación automatizada del árbol de URLs de OWASP ZAP a partir de las rutas y parámetros descubiertos durante la fase de fuzzing con ffuf, e interoperabilidad hacia SQLMap para transferir selectivamente endpoints parametrizados sospechosos.
- Módulo de autenticación automatizada y gestión de sesiones HTTP: inicio de sesión programático mediante solicitudes POST y renovación heurística de cookies de sesión para sostener el análisis en secciones restringidas durante escaneos prolongados.
- Filtrado heurístico y ejecución asíncrona de SQLMap: discriminación previa de URLs candidatas basada en la existencia de parámetros manipulables y ejecución desacoplada en segundo plano para evitar bloqueos del flujo principal.

b) Capa de Persistencia, Caché y API de Servicio:

- Ingesta, parseo sintáctico y normalización de salidas dispares: transformación de las respuestas heterogéneas (XML/JSON de ZAP, JSON estructurado de ffuf y registros de texto plano de SQLMap) hacia un esquema canónico consolidado en formato JSON (*resultado_unificado.json*).
- Motor de caché incremental y re-evaluación dirigida: indexación de hallazgos por identificador de endpoint y vector de ataque, permitiendo re-verificar con celeridad vulnerabilidades previamente confirmadas sin reiterar la ejecución exhaustiva del pipeline completo.
- Exposición de capacidades mediante arquitectura de servicio REST (FastAPI): despacho asíncrono de tareas de análisis mediante subprocesos en segundo plano (*BackgroundTasks*), monitoreo del ciclo de vida del escaneo en tiempo real y consulta histórica de auditorías.
- Capa de persistencia relacional híbrida: compatibilidad nativa tanto con SQLite para ejecuciones portátiles y pruebas locales, como con PostgreSQL para entornos de servicio multiusuario centralizados.

c) Interfaz de Usuario y Automatización Operativa:

- Interfaz web interactiva (Single Page Application desarrollada en React): panel de control para la parametrización de objetivos, visualización gráfica y tabular de vulnerabilidades consolidadas y consulta histórica de análisis.
- Generación automatizada de reportes técnico y ejecutivo: consolidación de resúmenes estructurados en formatos Markdown y JSON con matriz de riesgo global orientativa para audiencias técnicas y gerenciales.

- Asistencia en remediación mediante modelos de lenguaje: canal de consulta contextual a proveedores de inteligencia artificial (Groq/Gemini) para sugerir directrices de mitigación técnica específicas frente a los hallazgos confirmados.
- Aprovisionamiento reproducible multicontenedor: despliegue integral del orquestador, la aplicación objetivo de prueba y los motores de análisis mediante Docker Compose sobre una red puente aislada (*sec-net*).

9.2 Limitaciones

La declaración explícita de limitaciones constituye una salvaguarda metodológica indispensable en la ingeniería de seguridad de la información. Este ejercicio permite delimitar con precisión las condiciones bajo las cuales los resultados del orquestador son técnicamente reproducibles y válidos, evitando interpretaciones extrapoladas o falsas expectativas de cobertura en entornos productivos. Las restricciones identificadas se estructuran en tres dimensiones formales:

a) Delimitación del Entorno de Pruebas y Seguridad de la Infraestructura:

- Validación circunscrita al entorno de laboratorio controlado: las pruebas empíricas se ejecutaron exclusivamente sobre la aplicación deliberadamente vulnerable DVWA (Damn Vulnerable Web Application); no se realizaron auditorías sobre aplicaciones en producción reales ni contra infraestructuras protegidas por cortafuegos de aplicaciones web (WAF), balanceadores de carga o mecanismos anti-escaneo adaptativos (tales como desafíos CAPTCHA o bloqueo por frecuencia de peticiones), por lo que la extrapolación de los resultados a plataformas corporativas vivas no está empíricamente demostrada.
- Política de seguridad permisiva en la API REST de laboratorio: el prototipo implementado no incorpora autenticación de usuarios mediante tokens JWT u OAuth2, y opera con control de acceso cruzado abierto (CORS configurado para aceptar cualquier origen), decisión adoptada deliberadamente para simplificar la interacción en el laboratorio local pero que inhabilita su exposición directa en redes públicas no confiables.
- Carácter estrictamente orientativo del componente de IA: las directrices de mitigación emitidas por los modelos de lenguaje no fueron contrastadas sistemáticamente contra un panel de expertos ni validadas mediante pruebas de regresión en código real; deben tratarse como sugerencias preliminares de apoyo y nunca como instrucciones definitivas de remediación.

b) Restricciones de Comparabilidad Metodológica y Rigor Estadístico:

- Ausencia de experimento con grupo de control independiente para la medición de tiempos (H1): el ahorro de esfuerzo se evaluó mediante el contraste analítico entre la duración del pipeline automatizado (14 minutos) y los tiempos documentados en una sesión de validación y triaje sobre hallazgos ya detectados (50 minutos manuales sobre 125 minutos totales; véanse el Capítulo 17 y la Tabla 13), pero sin un experimento ciego en el que auditores humanos realicen descubrimiento desde cero, lo que condiciona la verificación empírica formal de la Hipótesis 1 (como se discute en la sección 14.2).
- Evaluación de cobertura basada en una corrida experimental única (H2): el análisis comparativo de cobertura individual frente a la combinación de herramientas (Tabla 9) se sustenta en una ejecución documentada y representativa, y no en un muestreo estadístico de múltiples repeticiones que permita calcular intervalos de confianza o varianzas, acotando el alcance demostrativo de la Hipótesis 2.
- Variabilidad intrínseca del escaneo dinámico y latencias de red: conforme a la amenaza a la confiabilidad descrita en la sección 10.5, factores como la latencia de la pila de red en Docker, la

conurrencia de hilos y los tiempos de respuesta del servidor web pueden introducir variaciones marginales en el número de solicitudes procesadas entre corridas sucesivas.

- Modelo heurístico simplificado de clasificación de riesgo: el reporte ejecutivo categoriza el riesgo global mediante una regla booleana simple (presencia de inyecciones SQL o alertas de severidad *High*), prescindiendo de esquemas cuantitativos normalizados de la industria como el estándar CVSS (Common Vulnerability Scoring System).
- Disponibilidad de artefactos crudos de benchmark y trazabilidad en repositorio: Las mediciones cronometradas y de estabilidad asociadas a la evaluación del impacto del caché incremental (Hipótesis 4, Tablas 10 y 11) se basan en los registros cuantitativos consolidados por los autores durante la experimentación de laboratorio del 6 de agosto de 2026. Debido a la política de exclusión de artefactos volátiles configurada en el archivo `.gitignore` del entorno de desarrollo (`output/raw/*`), los volcados JSON crudos intermedios (`resultado_sin_cache.json` y `resultado_con_cache.json`) no fueron preservados en el control de versiones del repositorio Git. Esta condición limita la auditabilidad retrospectiva independiente de dichos archivos crudos por parte de terceros, sustentándose la validez de H4 en las métricas cronometradas consolidadas en el presente informe.

c) Vulnerabilidades Fuera de Alcance por Diseño Arquitectónico (Scope Boundaries):

- Concentración en vectores analizables mediante firmas sintácticas: el orquestador focaliza su capacidad en fallas dinámicas detectables por patrones de inyección y análisis de anomalías en respuestas HTTP (inyecciones SQL, vectores reflejados de Cross-Site Scripting, inclusión local de archivos, omisión de cabeceras de seguridad y exposición de rutas no enlazadas).
- Exclusión explícita de vulnerabilidades de lógica de negocio (*Business Logic Flaws*): aquellas fallas que vulneran los flujos transaccionales legítimos de una aplicación (como omisión de etapas en pasarelas de pago, manipulación de estados de cuenta o inconsistencias en flujos de compra) escapan al modelo de análisis por firmas del escaneo dinámico y exigen necesariamente modelado funcional y auditoría manual.
- Exclusión de condiciones de carrera (*Race Conditions*) y control de acceso horizontal complejo: vectores basados en concurrencia temporal distribuida a nivel de microsegundos o violaciones complejas de control de acceso horizontal entre usuarios con idéntico nivel de privilegios demandan una orquestación semántica de permisos y sincronización que excede el diseño modular del pipeline desarrollado.

10. Metodología

10.1 Diseño de la Investigación

Se adopta un paradigma post-positivista con enfoque predominantemente cuantitativo en la medición de variables del sistema (cantidad de hallazgos, severidad, cobertura), combinado con un diseño de investigación tecnológica aplicada de tipo constructivo: el objetivo central no es contrastar una hipótesis mediante un experimento controlado con grupo de comparación, sino diseñar, construir y validar un artefacto de software contra criterios de aceptación predefinidos. El método de trabajo es iterativo e incremental: cada componente del pipeline se implementa, prueba y valida de forma independiente antes de ser integrado al sistema completo. Las pruebas de funcionamiento se realizan mediante la ejecución del pipeline contra DVWA, una aplicación web intencionalmente vulnerable utilizada como entorno de prueba (testbed), y los resultados se evalúan contra los criterios de validación definidos en la sección 10.6.

10.2 Fases del Proyecto

El proyecto se desarrolló en seis fases secuenciales. La primera fase consistió en la investigación de herramientas de seguridad web, donde se evaluaron las capacidades, formatos de salida y posibilidades de integración de diversas herramientas open source. La segunda fase abordó el diseño de la arquitectura del sistema, definiendo la estructura modular de paquetes, los flujos de datos y las interfaces entre componentes. La tercera fase fue la implementación del orquestador en Python, incluyendo los módulos de ejecución, escaneo, parseo y consolidación. La cuarta fase comprendió la integración con contenedores Docker, configurando el archivo docker-compose.yml para desplegar DVWA, ZAP y la aplicación orquestadora. La quinta fase se dedicó al desarrollo de parsers de resultados, implementando la normalización de las salidas heterogéneas de cada herramienta. La sexta fase consistió en las pruebas en entorno de laboratorio, ejecutando el pipeline completo contra DVWA y documentando los resultados obtenidos.

Una séptima fase, posterior a la planificación original, extendió el sistema hacia un servicio persistente: incorporó autenticación automática contra la aplicación objetivo, una capa de base de datos (SQLite/PostgreSQL) para historial de escaneos y caché incremental, una API REST (FastAPI) para ejecución asíncrona del pipeline, y un frontend web que consume dicha API para lanzar escaneos y visualizar resultados.

10.3 Variables

La variable independiente de la investigación es la automatización del proceso de análisis de seguridad mediante el orquestador. Se define operacionalmente como la ejecución coordinada de las herramientas OWASP ZAP (spider y escaneo activo), ffuf y SQLMap a través del pipeline implementado en Python, en contraposición a la ejecución manual e independiente de cada herramienta por parte de un analista.

Las variables dependientes son las siguientes:

1. Número de vulnerabilidades detectadas: cantidad total de hallazgos de seguridad identificados por el sistema durante una ejecución completa del pipeline, medida a partir del archivo JSON unificado generado por el módulo de consolidación.
2. Severidad de los hallazgos: clasificación de las vulnerabilidades detectadas según su nivel de riesgo (High, Medium, Low, Informational), tal como es reportada por las herramientas integradas, particularmente por OWASP ZAP.
3. Cobertura de análisis: cantidad de URLs únicas descubiertas y analizadas por el conjunto de herramientas, incluyendo las identificadas por el spider, las verificadas por el escaneo activo, las descubiertas por el fuzzing de directorios y las incorporadas como URL semilla o por el módulo de autenticación (ver nota de composición en la sección 13.1).

4. Tiempo de ejecución del pipeline: duración de cada etapa y del proceso completo, medible mediante instrumentación de timestamps al inicio y fin de cada etapa. Actualmente esta medición es parcial: cubre el mecanismo de caché de ffuf mediante `test_speed.py` y el pipeline completo mediante la comparación con/sin caché de la sección 14.3, y está pendiente de extenderse a una medición equivalente del proceso manual (ver 14.2).

Las variables de control comprenden el entorno objetivo (DVWA en su configuración por defecto), las versiones específicas de las herramientas utilizadas (imágenes Docker), las wordlists empleadas por ffuf, y los niveles de profundidad y riesgo configurados en SQLMap.

10.4 Instrumentos

Los instrumentos de la investigación corresponden a los componentes del sistema desarrollado. Los contenedores Docker proveen la infraestructura reproducible de ejecución. El pipeline en Python (Python Software Foundation, 2024) recolecta los datos coordinando cada herramienta y sus salidas crudas; los parsers especializados normalizan los resultados heterogéneos, y el módulo de consolidación los sintetiza en el archivo JSON unificado de análisis. Asimismo, el script `test_speed.py` actúa como instrumento de medición temporal para evaluar la caché de ffuf.

10.5 Amenazas a la Validez

- **Validez interna:** DVWA es un entorno diseñado deliberadamente para ser vulnerable, lo que puede facilitar artificialmente la detección de hallazgos respecto a una aplicación real con controles de seguridad activos.
- **Validez externa:** los resultados obtenidos sobre DVWA no son necesariamente generalizables a aplicaciones en producción con arquitecturas, frameworks o controles de seguridad distintos.
- **Confiabilidad:** no se asume estabilidad absoluta entre ejecuciones de ZAP y SQLMap. Las diferencias observadas en URLs y alertas frente a corridas previas (ver nota metodológica del capítulo 13) confirman empíricamente que la reproducibilidad exacta no está garantizada, reforzando la exigencia de registrar fecha y condiciones puntuales en cada reporte.

10.6 Criterios de Validación

Un componente del sistema se considera validado cuando cumple simultáneamente los siguientes criterios:

- **Funcionalidad:** la herramienta se ejecuta correctamente dentro del pipeline y produce una salida procesable por el parser correspondiente.
- **Reproducibilidad:** una ejecución posterior sobre el mismo objetivo debe arrojar resultados equivalentes en estructura y tipo de hallazgos, fijándose un umbral de coincidencia mínimo del 90%. Como se verificó en la sección 14.3 (Tabla 11), la coincidencia en alertas ZAP y vulnerabilidades SQLMap alcanzó el 100%, y la superficie de URLs el 96,3% (superando el umbral pese a la dinámica del spider señalada en 10.5).
- **Cobertura:** el sistema detecta vulnerabilidades pertenecientes a al menos tres categorías distintas del OWASP Top 10.
- **Consolidación:** todos los hallazgos de las herramientas individuales se integran correctamente en el archivo JSON unificado final.

11. Arquitectura Propuesta e Implementada

11.1 Descripción General

La arquitectura del sistema se organiza en cuatro capas, ilustradas en la Figura 1:

1. Orquestador Python: coordina la ejecución del pipeline (spider → ffuf → inyección de rutas en ZAP → escaneo activo → SQLMap), gestiona autenticación y sesión, y consolida resultados.
2. Herramientas de escaneo: OWASP ZAP (spider y escaneo activo), ffuf (fuzzing de directorios) y SQLMap (inyección SQL), cada una encapsulada en su propio módulo (ver Figura 2).
3. Capa de persistencia: base de datos SQLite o PostgreSQL que almacena historial de escaneos, caché incremental de ffuf y SQLMap, y URLs marcadas para re-testeo automático.
4. Capa de servicio: una API REST (FastAPI) que expone el pipeline de forma asíncrona con seguimiento de progreso, y un frontend web que consume esa API para lanzar escaneos y visualizar resultados, incluyendo sugerencias de mitigación obtenidas mediante un flujo de n8n.

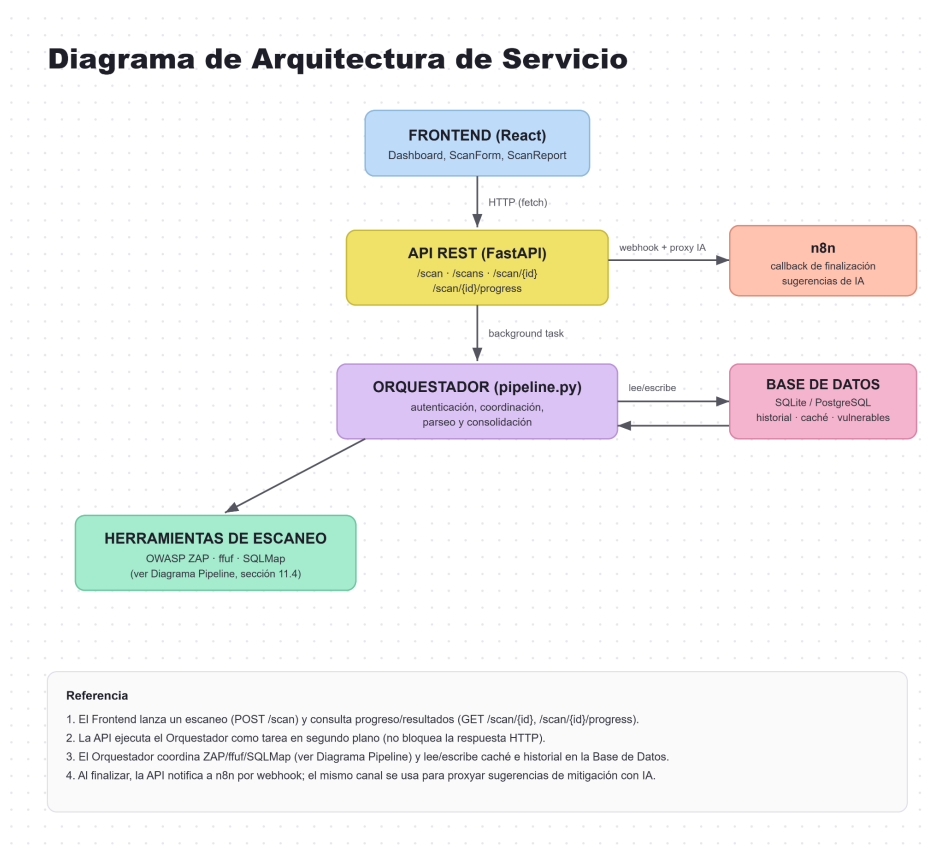


Figura 1. Diagrama de Arquitectura de Servicio. Elaboración propia (véase versión ampliada en el Anexo H).

11.2 Componentes y Sus Roles

Tabla 2. Componentes del sistema y su función en el pipeline

Componente	Contenedor	Función en el pipeline
DVWA	dvwa	Aplicación objetivo de las pruebas
OWASP ZAP	zap	Spider, escaneo activo y generación de reportes vía API REST
Orquestador Python	security-app	Coordinación del pipeline, autenticación automática, ejecución de ffuf y SQLMap, parseo y consolidación
API REST	security-app	Expone el pipeline como servicio asíncrono (/scan, /scans, /scan/{id}, /scan/{id}/progress)
Base de datos	db (SQLite embebida o PostgreSQL)	Persistencia de historial de escaneos, caché incremental de ffuf/SQLMap y URLs marcadas para re-testeo

Nota. Elaboración propia.

La Figura 2 detalla cómo el orquestador coordina específicamente a las tres herramientas de escaneo y el enriquecimiento cruzado entre ellas.

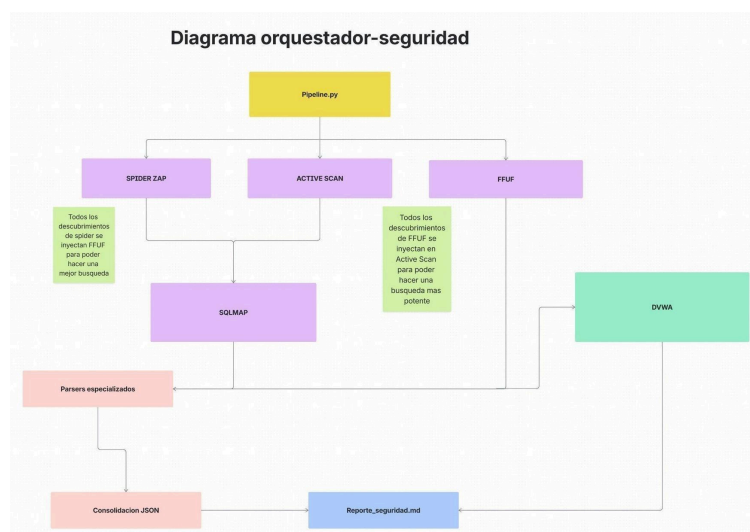


Figura 2. Diagrama Orquestador-Seguridad: coordinación y enriquecimiento cruzado entre ZAP, ffuf y SQLMap. Elaboración propia (véase versión ampliada en el Anexo H).

11.3 Red Docker y Aislamiento

Todos los servicios se ejecutan dentro de una red virtual Docker gestionada por Docker Compose. Esta configuración garantiza el aislamiento del tráfico interno entre contenedores y permite la resolución de nombres de servicio: los contenedores se comunican entre sí por nombre (por ejemplo, el orquestador se conecta a ZAP mediante el hostname zap y a DVWA mediante el hostname dvwa). Los puertos se exponen al host únicamente para los servicios que requieren acceso externo durante el desarrollo. La API habilita CORS para permitir que el frontend, ejecutado fuera de la red interna de Docker, consuma sus endpoints durante el desarrollo.

11.4 Flujo de Datos Detallado

El ciclo de vida completo de un análisis de seguridad en el sistema sigue la secuencia representada en la Figura 3:

1. El orquestador establece sesión autenticada contra la aplicación objetivo (login automático y configuración del nivel de seguridad) y limpia la sesión previa de ZAP.
2. Ejecuta el spider de ZAP contra la URL objetivo, que recorre la aplicación web descubriendo las URLs accesibles.
3. Ejecuta ffuf contra la URL base, consultando primero el historial en base de datos para descartar palabras de wordlist ya testeadas contra ese objetivo.
4. Las rutas descubiertas por ffuf se inyectan en la sesión de ZAP antes del paso siguiente.
5. Se ejecuta el escaneo activo de ZAP, que ataca tanto las URLs descubiertas por su propio spider como las rutas aportadas por ffuf.
6. Se combinan las URLs del spider y de ffuf, se filtran las candidatas con parámetros o rutas interactivas, se descartan las que ya están cacheadas sin hallazgos previos y se conserva siempre el re-testeo de las marcadas como vulnerables en ejecuciones anteriores; SQLMap se ejecuta en segundo plano sobre esa lista final, con la sesión de autenticación renovada para evitar su expiración.
7. Los resultados crudos de cada herramienta son procesados por parsers especializados que normalizan las estructuras de datos y eliminan duplicados.
8. El módulo de consolidación unifica los resultados de todos los parsers en un único archivo JSON estructurado, que alimenta tanto el reporte técnico como el reporte ejecutivo.

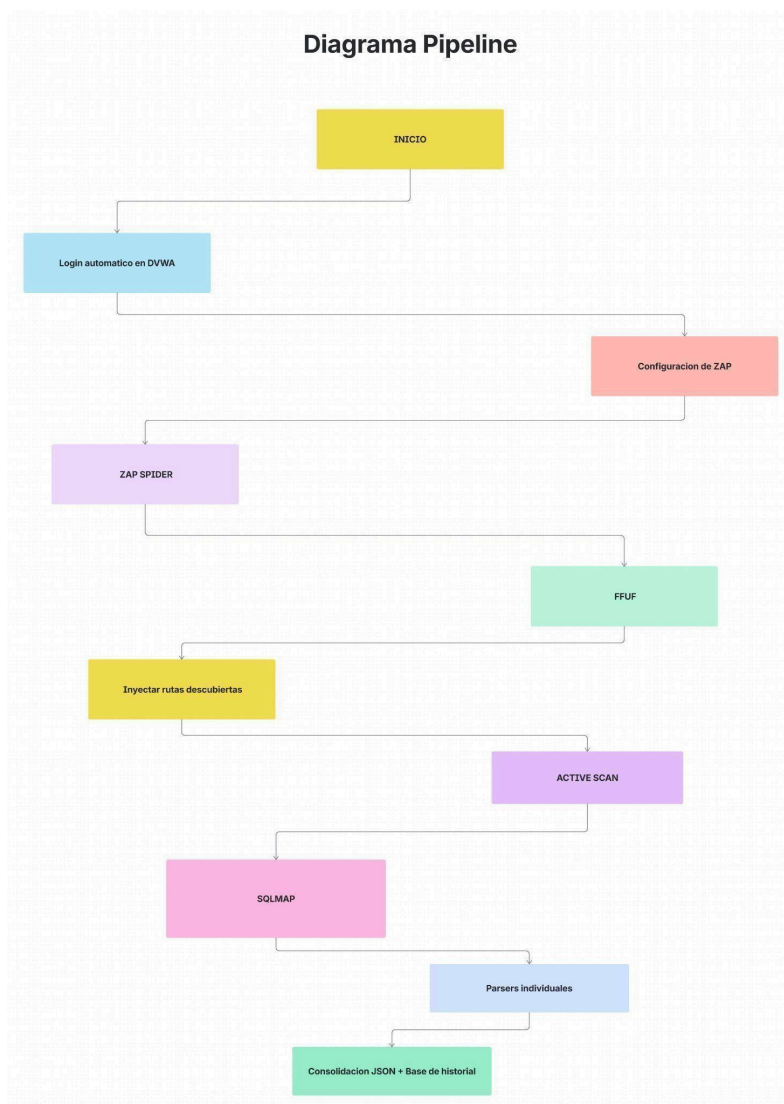


Figura 3. Diagrama de Pipeline de ejecución. Elaboración propia (véase versión ampliada en el Anexo H).

11.5 Diagrama de Secuencia — Ciclo de Vida de POST /scan

Las Figuras 1 a 3 documentan la arquitectura estática del sistema y el flujo interno del pipeline de escaneo, pero no el comportamiento asíncrono de la API frente al cliente. La Figura 4 completa esa descripción con un diagrama de secuencia UML del ciclo de vida completo de una petición POST /scan, desde que el frontend la dispara hasta que la API notifica su finalización a n8n.

El diagrama distingue explícitamente entre la respuesta inmediata de la API (202 Accepted con el scan_id, antes de que el pipeline haya siquiera comenzado a ejecutarse) y el trabajo en segundo plano que realiza ejecutar_pipeline_segundo_plano(): mientras este último coordina secuencialmente a ZAP, ffuf y SQLMap y lee/escribe en la base de datos, el frontend consulta el progreso mediante polling periódico a GET /scan/{id}/progress, sin bloquear ni depender de que la conexión HTTP original permanezca abierta. Esta separación entre la respuesta síncrona y la ejecución asíncrona es la que permite que el pipeline —cuya duración se documenta en la sección 14.3 entre 4 y 14 minutos según el estado del caché— no comprometa la responsividad de la API ni del frontend.

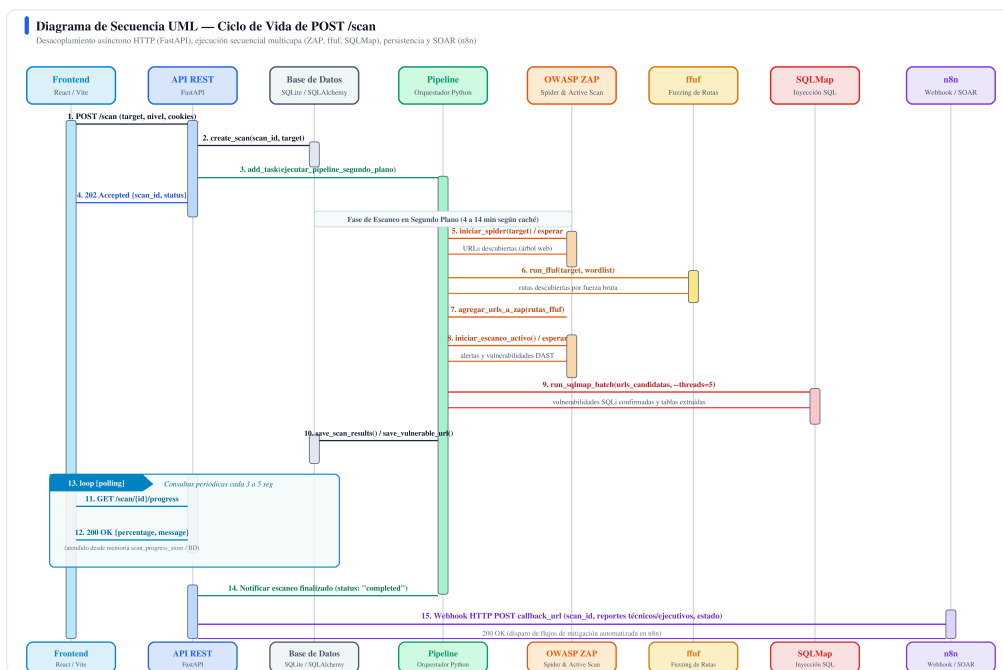


Figura 4. Diagrama de secuencia UML — ciclo de vida de POST /scan. Elaboración propia (véase versión ampliada en el Anexo H).

11.5.1 Detalle de la Secuencia de Interacción

El ciclo asíncrono representado en el diagrama comprende las siguientes etapas:

1. **Frontend → API (POST /scan):** el usuario inicia un escaneo desde el frontend enviando una petición HTTP POST /scan a FastAPI, incluyendo la URL objetivo y los parámetros de análisis.
2. **API → Base de datos (crear scan_id):** la API genera un identificador único (scan_id) y registra el escaneo en la base de datos junto con el objetivo, garantizando persistencia y trazabilidad.
3. **API → Pipeline (ejecutar en segundo plano):** la API despacha ejecutar_pipeline_segundo_plano() como tarea en segundo plano. La API no espera la conclusión del escaneo para responder, desacoplando el tiempo de escaneo del ciclo de vida HTTP.
4. **API → Frontend (202 Accepted + scan_id):** la API responde inmediatamente con código HTTP 202 Accepted y el scan_id. El escaneo continúa ejecutándose de forma asíncrona; el cliente solo recibe el identificador para consultar el estado.
5. **Pipeline → ZAP (ZAP Spider):** el pipeline inicia el spidering de OWASP ZAP para rastrear la aplicación web y recolectar las URLs accesibles del objetivo.
6. **Pipeline → ffuf (ffuf):** tras el spider, ffuf ejecuta un análisis de fuerza bruta mediante wordlists para descubrir rutas y directorios no vinculados directamente.
7. **Pipeline → ZAP (inyectar rutas descubiertas):** las nuevas rutas halladas por ffuf son incorporadas al árbol de sitios de ZAP, ampliando la superficie de ataque.
8. **Pipeline → ZAP (ZAP Active Scan):** con las rutas consolidadas, ZAP ejecuta el escaneo activo atacando tanto las rutas del spider como las aportadas por ffuf para identificar vulnerabilidades.
9. **Pipeline → SQLMap:** concluido el Active Scan, el pipeline extrae las URLs candidatas y despacha SQLMap en segundo plano para corroborar vulnerabilidades de inyección SQL.

10. **Pipeline → Base de datos (guardar resultados):** durante la ejecución, el sistema persiste de forma incremental el estado y los hallazgos en la base de datos para mantener el historial.
11. **Frontend → API (GET /scan/{id}/progress):** de forma desacoplada, el frontend consulta periódicamente el estado del escaneo enviando peticiones de polling a GET /scan/{id}/progress con el scan_id correspondiente.
12. **API → Frontend (porcentaje de progreso):** la API responde con el avance relativo del pipeline (percentage: 100 y "Análisis Completado" al finalizar). El término "porcentaje" refleja estrictamente el avance secuencial de etapas del orquestador, no una métrica de cobertura ni de vulnerabilidades detectadas.
13. **Ciclo de Polling Periódico:** las consultas y respuestas de progreso se reiteran a intervalos regulares sin mantener abierta la conexión original de POST /scan.
14. **Pipeline → API (escaneo finalizado):** al concluir todas las herramientas integradas (ZAP Spider, ffuf, ZAP Active Scan y SQLMap), el pipeline notifica a la API la finalización del trabajo.
15. **API → n8n (webhook HTTP):** finalmente, la API dispara una notificación vía webhook HTTP a n8n para informar la culminación del escaneo e iniciar los flujos automatizados de sugerencias de mitigación.

12. Implementación Técnica

12.1 Infraestructura Docker

El sistema se define en un archivo `docker-compose.yml` (Docker Inc., 2024) que especifica los servicios `dvwa` (la aplicación objetivo, basada en la imagen `vulnerables/web-dvwa`, DVWA Project, 2024), `zap` (el escáner OWASP ZAP, ejecutado en modo `daemon` con su API REST habilitada en el puerto 8090), `db` (base de datos PostgreSQL para historial y caché) y `app` (el orquestador Python, que se construye a partir del `Dockerfile` del proyecto y expone además la API REST en el puerto 8000), además del servicio `n8n` utilizado para las sugerencias de mitigación asistidas por IA. La configuración de dependencias mediante la directiva `depends_on` asegura que ZAP, DVWA y la base de datos se encuentren disponibles antes de que el orquestador inicie su ejecución. El frontend se desarrolla como un proyecto npm independiente, fuera de esta definición de Docker Compose, y se conecta a la API mediante HTTP. El contenido íntegro del archivo se transcribe en el Anexo A.

El contenedor del orquestador monta el directorio del proyecto como un volumen (`./app`), lo que permite que los archivos de salida generados durante la ejecución del pipeline sean accesibles directamente desde el sistema de archivos del host.

12.2 Pipeline de Ejecución

El pipeline de ejecución se implementa en el módulo `app/workflow/pipeline.py`, que expone dos funciones principales: `run_security_pipeline()` y `run_parser_pipeline()`. La primera constituye el punto de entrada del proceso de escaneo: gestiona la autenticación, ejecuta en orden spider de ZAP, ffuf, inyección de rutas descubiertas, escaneo activo de ZAP y SQLMap en segundo plano, devolviendo un diccionario con los datos crudos de las cuatro fuentes. La segunda función invoca los parsers especializados y consolida los resultados normalizados.

12.3 Módulo Runner

El módulo `app/runners/exec.py` implementa la función `run_command()`, que actúa como capa de abstracción para la ejecución de comandos del sistema operativo mediante `subprocess.run()`. El parámetro `timeout` tiene un valor por defecto de 60 segundos a nivel del runner genérico, pero cada scanner puede sobrescribirlo: SQLMap recibe explícitamente el valor `SQLMAP_TIMEOUT` centralizado en `settings.py` (300 segundos), por lo que la diferencia entre ambos valores es una configuración deliberada por herramienta y no una inconsistencia entre secciones.

12.4 Módulo de Escaneo ZAP

El módulo `app/scanners/zap.py` encapsula toda la comunicación con la API REST de OWASP ZAP (OWASP Foundation, 2024): `iniciar_spider()`, `esperar_spider()` y `obtener_urls()` controlan el crawling inicial; `iniciar_escaneo_activo()` y `esperar_escaneo_activo()` configuran intensidad y umbral de alertas; `obtener_reporte_json()` descarga el reporte completo; `configurar_autenticacion()` registra la cookie de sesión para escaneos autenticados; `agregar_urls_a_zap()` inyecta en el árbol de sitios de ZAP las rutas descubiertas por ffuf; y `limpiar_sesion_zap()` reinicia la sesión de ZAP al comienzo de cada ejecución.

12.5 Módulo de Escaneo ffuf

El módulo `app/scanners/ffuf.py` implementa la función `run_ffuf()` (ffuf Project, 2024). Antes de ejecutar el fuzzer, consulta la base de datos por las palabras de la wordlist ya testeadas contra el objetivo, genera una wordlist temporal solo con las palabras nuevas, y marca la ejecución como `skipped` si no hay palabras nuevas para probar. El comando se configura con `-u`, `-w`, `-mc 200,302` y `-of json / -o`. Al finalizar, las palabras efectivamente probadas se persisten en la base de datos.

12.6 Módulo de Escaneo SQLMap

El módulo `app/scanners/sqlmap.py` implementa la detección de inyecciones SQL (SQLMap Project, 2024). La función `filtrar_urls_con_parametros()` prioriza las URLs con parámetros GET y rutas `.php` de formularios interactivos conocidos, excluyendo rutas estáticas o de configuración. El comando de SQLMap se ejecuta con `--batch`, `--flush-session`, `--forms`, `--dbms=MySQL`, `--level=1`, `--risk=3`, `--threads=5`, `--technique=BEUST` y `-o`. La bandera `--smart` fue retirada intencionalmente del pipeline debido a que, en aplicaciones como DVWA, las respuestas base homogéneas ante cualquier parámetro provocaban que la heurística de descarte rápido omitiera en silencio endpoints vulnerables. La función `run_sqlmap_batch()` separa las URLs candidatas en tres grupos según el historial en base de datos: las marcadas como vulnerables se re-testean siempre; las ya analizadas sin hallazgos se omiten; las nuevas se encolan. El escaneo se ejecuta en segundo plano mediante un script bash generado dinámicamente. Asimismo, la función `run_sqlmap()` admite el parámetro `sqlmap_level`, el cual modula el alcance de explotación en tres modalidades: `basic` (detección y confirmación con el comando `base`), `fast_evidence` (incorpora `--dbs`, `--tables` y `--current-user` para recabar evidencia del esquema sin volcar datos) y `full_dump` (añade `--dump` para extraer íntegramente las tablas vulnerables confirmadas).

12.7 Sistema de Parseo

El sistema de parseo se implementa en el paquete `app/parsers/` y consta de tres módulos especializados. `zap_parser.py` normaliza los resultados del spider y aplana las alertas del reporte de ZAP eliminando duplicados por clave (`url`, `alerta`). `ffuf_parser.py` filtra las rutas con códigos HTTP 200 y 302 y elimina duplicados por URL.

El parser de SQLMap, `sqlmap_parser.py`, funciona mediante la búsqueda de un conjunto de patrones de texto dentro de la salida estándar (`stdout`) capturada de cada ejecución. La función `parsear_sqlmap()` recorre la lista de resultados y considera una URL como vulnerable cuando su `stdout` contiene, en minúsculas, al menos una de las siguientes cadenas: “is vulnerable”, “is injectable”, “appears to be”, “vulnerability:”, “sql injection”, “payload:”, “type:”, “database names are:”, “current user is:”, “fetched data logged to text files” y “available databases”. Es, en consecuencia, un parser de coincidencia de texto sobre salida en lenguaje natural, no un parser que se apoye en el código de retorno del proceso ni en una salida estructurada en formato JSON (como la que expone la API REST oficial de SQLMap a través de `sqlmapapi.py`). Esto lo hace dependiente del formato exacto de los mensajes que SQLMap imprime por consola, un formato que no está garantizado entre versiones de la herramienta: un cambio de redacción en una futura versión de SQLMap podría hacer que una URL efectivamente vulnerable no sea reconocida como tal (falso negativo), sin que el pipeline reporte ningún error, ya que el parser no distingue entre “no se encontraron estos patrones porque no hay vulnerabilidad” y “no se encontraron estos patrones porque el mensaje cambió de forma”. Elia et al. (2010) documentan precisamente este tipo de riesgo al comparar herramientas de detección de inyección SQL basadas en distintos mecanismos de análisis de salida. Una alternativa más robusta, no implementada en esta versión, sería inspeccionar el código de retorno del proceso e interactuar directamente con la API REST (`sqlmapapi.py`), la cual genera respuestas nativas en formato JSON estructurado, evitando así la dependencia de cadenas de texto en lenguaje natural impresas en la consola.

Los tres parsers comparten un patrón de diseño consistente: reciben un diccionario o lista cruda, aplican transformaciones y filtrado, eliminan duplicados e incluyen manejo de excepciones (`KeyError`, `TypeError`) que devuelven estructuras vacías en caso de error, evitando que un fallo en el parseo de una herramienta interrumpa el pipeline completo.

12.8 Consolidación de Resultados

El módulo `app/utils/results.py` implementa la función `consolidar_resultados()`, que recibe los diccionarios normalizados de spider, ZAP, ffuf y SQLMap y genera una estructura unificada con un bloque de resumen (conteos agregados) y los resultados completos de cada herramienta. La función `resultados_prueba_json()` persiste el resultado consolidado en un archivo JSON en `output/raw/`.

12.9 Autenticación Automática

El sistema implementa un flujo de login programático contra DVWA (`establecer_sesion_automatica()`): obtiene el token CSRF del formulario de login, envía las credenciales, fija el nivel de seguridad de la aplicación y verifica el éxito del login. Si el login falla, el sistema intenta inicializar automáticamente la base de datos de la aplicación objetivo mediante `setup.php` antes de reintentar. La cookie de sesión obtenida se reutiliza para autenticar a ZAP, ffuf y SQLMap, y se renueva antes de ejecutar SQLMap para evitar que expire durante el escaneo activo de ZAP.

12.10 API REST y Frontend

El módulo `api.py` (FastAPI) expone el pipeline como servicio: `POST /scan` lanza un escaneo en segundo plano, admitiendo los parámetros opcionales `sqlmap_level: str` (`basic`, `fast_evidence` o `full_dump`) y `clean_cache: bool` (el cual purga el historial en base de datos para forzar una ejecución limpia sin reutilización de caché); `GET /scan/{id}/progress` permite consultar el porcentaje de avance; `GET /scans` lista el historial persistido; `GET /scan/{id}` devuelve el detalle de un escaneo puntual; `DELETE /scan/{id}` realiza un borrado lógico. Al finalizar un escaneo, la API notifica su finalización mediante un webhook HTTP a un flujo de n8n. El frontend, desarrollado en React, consume estos endpoints para ofrecer un panel de dominios escaneados, un formulario de lanzamiento de escaneos, una tabla de vulnerabilidades y un componente de sugerencias de mitigación que consulta un endpoint adicional de análisis asistido por IA expuesto a través del router de n8n. El código fuente integral de estos componentes y el registro audiovisual de demostración en tiempo real se encuentran documentados para su verificación en el Anexo I.

Nota sobre pruebas. Los scripts `test_parsers.py`, `test_speed.py`, `test_zap.py` y `test_sqlmap.py` son scripts de verificación manual (ejecutan una función y muestran el resultado por consola), no una suite de pruebas unitarias automatizadas con aserciones.

13. Resultados Obtenidos

Los resultados de este capítulo corresponden a la ejecución del pipeline completo contra DVWA (DVWA Project, 2024) registrada el 5 de agosto de 2026 a las 19:53 (hora local del entorno), y son verificables directamente contra el archivo `reporte_seguridad.md` conservado en el directorio `orquestador-seguridad/output/reports/` y respaldado como evidencia primaria en `docs/evidencia/reportes_seguridad.md`. Cabe aclarar que, debido a la política de exclusión de archivos temporales en el entorno de desarrollo (regla `output/raw/*` en `.gitignore`), los registros crudos intermedios generados por las herramientas en dicha pasada no fueron preservados en el control de versiones, constituyendo el reporte consolidado el artefacto documental maestro de auditoría para esta ejecución.

Nota metodológica sobre la fuente de estos datos. Una versión preliminar de este capítulo citaba cifras (61 URLs, 55 del spider, 56 alertas de ZAP) extraídas de un archivo `test_resultado_final.json` que no se conserva en el repositorio del proyecto, por lo que no pudieron verificarse contra ninguna corrida real disponible. Esta versión reemplaza esas cifras por los datos de la ejecución más reciente y verificable del pipeline (5 de agosto de 2026), trazable directamente a `reporte_seguridad.md`. Cabe aclarar que ZAP y SQLMap son herramientas de escaneo dinámico cuyo comportamiento no es estrictamente determinístico entre corridas —el orden de exploración del spider, los tiempos de respuesta del servidor y el estado de la caché incremental pueden variar el número exacto de URLs y alertas detectadas de una ejecución a otra sobre el mismo objetivo—, tal como se declara en la amenaza a la validez de Confiabilidad (sección 10.5). En consecuencia, las cifras de este capítulo deben leerse como el resultado de una ejecución puntual, documentada y trazable, y no como una constante reproducible con exactitud aritmética entre corridas.

Evidencia adicional de esta variabilidad: esta misma corrida del 5 de agosto ofrece un ejemplo concreto del fenómeno descrito arriba. Una ejecución previa registrada el 2 de agosto de 2026 había confirmado inyección SQL tanto en el parámetro `id` de `/vulnerabilities/sqli/` como en el parámetro `username` de `/vulnerabilities/brute/` (8 hallazgos individuales, 4 técnicas × 2 endpoints). En la corrida del 5 de agosto documentada en este capítulo, SQLMap solo confirmó el segundo punto (`/vulnerabilities/brute/`, parámetro `username`, 4 hallazgos individuales); el endpoint `/vulnerabilities/sqli/` no volvió a reportarse como vulnerable en esta pasada, pese a no haberse modificado el objetivo entre ambas corridas. Spider, ZAP y ffuf se mantuvieron estables entre ambas ejecuciones. Este contraste entre dos corridas reales —no hipotético— es la evidencia más directa disponible en este informe de que los resultados de SQLMap pueden variar entre ejecuciones sobre el mismo objetivo sin cambios de por medio, y es retomado con un tercer punto de datos en la sección 14.3.

13.1 Resumen de Hallazgos

Tabla 3. Resumen de hallazgos — ejecución sobre DVWA del 5 de agosto de 2026

Indicador	Valor
Total de URLs únicas descubiertas y analizadas	34
URLs descubiertas por el Spider	24
Alertas de seguridad reportadas por ZAP	37
Rutas descubiertas por ffuf	8
Hallazgos individuales reportados por SQLMap	4 (4 técnicas × 1 endpoint vulnerable)

Nota. Datos extraídos de `reporte_seguridad.md` (sección Estadísticas Generales). El total de 34 URLs únicas no es la simple suma de spider (24) y ffuf (8): las 2 URLs restantes corresponden a la URL semilla del objetivo (`http://dvwa/`) y a una URL derivada del proceso de autenticación automática (`http://dvwa/login.php` con parámetros de sesión), incorporadas al conjunto unificado antes de la deduplicación por el módulo `consolidar_resultados()` del pipeline.

13.2 Vulnerabilidades Detectadas por OWASP ZAP

El escaneo activo de OWASP ZAP reportó un total de 37 alertas de seguridad. La Tabla 4 sintetiza los tipos de alerta detectados y su cantidad de ocurrencias.

Cómo leer la columna Severidad. Cada valor sigue el formato Riesgo (Confianza) que expone el campo `riskdesc` de la API de ZAP: el primer término (High, Medium, Low o Informational) es el nivel de riesgo de la vulnerabilidad, y el término entre paréntesis es el nivel de confianza — qué tan seguro está ZAP de que el hallazgo es real y no un falso positivo — (también High, Medium, Low, o Confirmed). Son dos escalas independientes: un hallazgo “Low (High)”, como Server Leaks Version Information en la tabla siguiente, es de bajo riesgo pero con alta confianza de que el hallazgo es correcto; en cambio un “Medium (Medium)” es de riesgo medio con una confianza también media, es decir, con mayor probabilidad relativa de ser un falso positivo. La confianza no debe confundirse con la severidad: un hallazgo de alto riesgo pero baja confianza amerita revisión manual antes de tratarse como confirmado, mientras que uno de bajo riesgo y alta confianza puede priorizarse con más certeza aunque su impacto sea menor.

Tabla 4. Vulnerabilidades detectadas por OWASP ZAP, por tipo

Vulnerabilidad	Severidad	Ocurrencias	URL de ejemplo
Content Security Policy (CSP) Header Not Set	Medium (High)	5	http://dvwa/about.php
Directory Browsing	Medium (Medium)	5	http://dvwa/docs/
Missing Anti-clickjacking Header	Medium (Medium)	5	http://dvwa/about.php
Server Leaks Version Information ("Server" header)	Low (High)	5	http://dvwa
X-Content-Type-Options Header Missing	Low (Medium)	5	http://dvwa/dvwa/css/login.css
User Agent Fuzzer	Informational (Medium)	5	http://dvwa/about.php
In Page Banner Information Leak	Low (High)	2	http://dvwa/sitemap.xml
Information Disclosure - Debug Error Messages	Low (Medium)	2	http://dvwa/instructions.php
HTTP Only Site	Medium (Medium)	1	http://dvwa/
Authentication Request Identified	Informational (High)	1	http://dvwa/login.php
Information Disclosure - Suspicious Comments	Informational (Medium)	1	http://dvwa/setup.php

Nota. Datos extraídos de reporte_seguridad.md (sección Alertas de Seguridad: OWASP ZAP). La severidad reproduce el campo riskdesc de ZAP en formato «Nivel de Riesgo (Nivel de Confianza)».

La distribución de las 37 alertas por nivel de severidad base es la siguiente: Medium 16 (43,2%), Low 14 (37,8%) e Informational 7 (18,9%). No se registraron alertas de severidad High en esta ejecución del escaneo pasivo/activo de ZAP; los hallazgos de severidad alta de esta entrega provienen de SQLMap (sección 13.4).

13.3 Rutas Descubiertas por ffuf

Tabla 5. Rutas descubiertas por fuzzing de directorios (ffuf)

Ruta Descubierta	Input (Wordlist)	Código HTTP
http://dvwa/about.php	about.php	200
http://dvwa/index.php	index.php	302
http://dvwa/instructions.php	instructions.php	200
http://dvwa/login.php	login.php	200
http://dvwa/logout.php	logout.php	302
http://dvwa/phpinfo.php	phpinfo.php	302
http://dvwa/security.php	security.php	302
http://dvwa/setup.php	setup.php	200

Nota. Datos extraídos de reporte_seguridad.md (sección Rutas descubiertas por ffuf).

13.4 Análisis Automatizado con SQLMap

SQLMap se ejecutó sobre las URLs con parámetros descubiertas por el spider y por ffuf. En esta corrida se confirmó como vulnerable el parámetro username del formulario de fuerza bruta (/vulnerabilities/brute/), mediante las cuatro técnicas configuradas (--technique=BEUST), lo que totaliza los 4 hallazgos individuales del resumen consolidado. El payload real registrado para la técnica boolean-based blind fue:

```
username=SCVZ' RLIKE (SELECT (CASE WHEN (4201=4201) THEN 0x5343565a ELSE 0x28 END))-- RmAL&password=Fqv&Login=Login
```

Tabla 6. Endpoints confirmados como vulnerables por SQLMap — corrida del 5 de agosto de 2026

Endpoint	Parámetro	Técnicas que confirmaron la inyección
/vulnerabilities/brute/	username	Boolean-based blind (RLIKE), Error-based (EXTRACTVALUE), Time-based blind (SLEEP), UNION query (8 columns)

Nota. Datos extraídos de reporte_seguridad.md (sección Detalles Técnicos: Inyecciones SQL).

Tabla 7. Datos extraídos a través de SQL Injection – corrida del 5 de agosto de 2026

ID de Usuario	Usuario	Contraseña (hash MD5)	Apellido	Nombre
1	admin	5f4dcc3b5aa765d61d8327deb882cf99	admin	admin
2	gordonb	e99a18c428cb38d5f260853678922e03	Brown	Gordon
3	1337	8d3533d75ae2c3966d7e0d4fcc69216b	Me	Hack
4	pablo	0d107d09f5bbe40cade3de5c71e9e9b7	Picasso	Pablo
5	smithy	5f4dcc3b5aa765d61d8327deb882cf99	Smith	Bob

Nota. Datos extraídos de reporte_seguridad.md (sección Volcado de Tabla: dvwa.users). Además, se evitaron columnas como «avatar», «last_login» o «failed_login» debido a que no aportan información relevante para el análisis presentado en este informe. Los valores mostrados son las credenciales por defecto de DVWA, una aplicación deliberadamente vulnerable; el criterio de manejo responsable de datos extraídos descrito en el capítulo 16 (principio 1.6 del código ACM) rige para cualquier uso de esta técnica sobre un objetivo real.

A partir de la explotación exitosa, SQLMap identificó el motor de base de datos como MySQL/MariaDB, enumeró la base de datos activa (dvwa) y extrajo el contenido de las tablas users (5 registros) y guestbook (1 registro), replicando el conjunto de datos por defecto de DVWA. Los timestamps last_login de la tabla users (2026-08-05 19:50:13) coinciden con el horario de esta corrida, confirmando que el volcado corresponde a una extracción en vivo y no a datos de ejemplo reutilizados. El volcado completo está disponible en `reporte_seguridad.md`.

Sobre el endpoint `/vulnerabilities/sqli/` (parámetro `id`): en la corrida del 2 de agosto de 2026 este endpoint también se había confirmado como vulnerable, con las mismas cuatro técnicas. En la corrida del 5 de agosto no fue reportado por SQLMap. No se modificó el código de la aplicación objetivo entre ambas fechas; la explicación más plausible es que, en esta pasada puntual, el conjunto de URLs con parámetros que el spider y ffuf le entregaron a SQLMap no incluyó una variante de `/vulnerabilities/sqli/` con un parámetro `id` explícito en la forma que SQLMap necesita para reconocerlo como candidato, o bien el mecanismo de re-testeo priorizó otro orden de ataque dentro del tiempo disponible. Esta explicación se retoma y refuerza con un tercer punto de datos en la sección 14.3.

13.5 Análisis por Categoría OWASP Top 10

Tabla 8. Hallazgos clasificados por categoría OWASP Top 10

Categoría OWASP	Vulnerabilidades detectadas	Herramienta
A01:2021 – Broken Access Control	Directory Browsing (5 instancias); rutas administrativas (<code>/setup.php</code> , <code>/security.php</code>) descubiertas por ffuf	ZAP, ffuf
A03:2021 – Injection	SQL Injection confirmada en <code>/vulnerabilities/brute/</code> (<code>username</code>); confirmada también en <code>/vulnerabilities/sqli/</code> (<code>id</code>) en la corrida previa del 2 de agosto de 2026	SQLMap
A05:2021 – Security Misconfiguration	CSP Header Not Set, Missing Anti-clickjacking Header, X-Content-Type-Options Missing, Server Leaks Version Information, HTTP Only Site	ZAP

Nota. Elaboración propia.

El sistema detectó vulnerabilidades pertenecientes a tres categorías distintas del OWASP Top 10 en esta ejecución, cumpliendo el criterio de cobertura mínima definido en la sección 10.6.

14. Discusión

Los resultados obtenidos confirman que es técnicamente viable implementar un sistema de orquestación de herramientas de seguridad web utilizando exclusivamente tecnologías open source y un desarrollo propio en Python. El pipeline automatizado logró detectar vulnerabilidades en múltiples categorías del OWASP Top 10, y el reporte técnico generado el 5 de agosto de 2026 (`output/reports/reporte_seguridad.md`) constituye evidencia de una ejecución real contra el entorno DVWA, con hostnames reales del entorno Docker en lugar de datos de ejemplo.

El sistema evolucionó considerablemente respecto al diseño original de tres módulos independientes (ZAP, ffuf, SQLMap) hacia una arquitectura de servicio: una API REST (`api.py`, FastAPI) que ejecuta el pipeline en segundo plano, expone el progreso del escaneo en tiempo real, persiste el historial de análisis en base de datos y notifica su finalización mediante webhooks a un flujo de n8n. Sobre esta API se construyó además un frontend que consume estos endpoints. Esta extensión responde directamente a PI4 (extensibilidad y reproducibilidad): la separación entre orquestador, API y frontend permitió incorporar estas capas sin modificar la lógica interna del pipeline.

14.1 Contraste de cobertura: uso individual frente a combinado (H2)

El pipeline actual no solo integra los hallazgos de las tres herramientas al final del proceso, sino que los cruza durante la ejecución: las rutas descubiertas por ffuf se inyectan en la sesión de ZAP antes de iniciar el escaneo activo, de modo que ZAP ataca tanto las URLs descubiertas por su propio spider como las descubiertas por ffuf; y SQLMap solo recibe URLs candidatas porque el spider y ffuf las aportaron previamente. La Tabla 9 descompone, a partir de esta misma ejecución, qué detecta cada herramienta por separado frente al resultado del pipeline combinado.

Tabla 9. Cobertura por herramienta en solitario frente al pipeline combinado (misma ejecución del 5 de agosto de 2026)

Herramienta / Método	URLs analizadas	Hallazgos	Cobertura OWASP Top 10	Limitación de operar en solitario
ZAP (spider + active scan)	24 (su propio spider)	37 alertas	A05:2021	No descubre rutas no enlazadas de forma proactiva; no profundiza en inyección SQL
ffuf	8 (wordlist)	0 (solo identifica códigos HTTP, no clasifica vulnerabilidades)	A01:2021 (indirecta)	No evalúa el contenido de las respuestas ni ejecuta lógica de detección
SQLMap en aislamiento total	0 (requiere URLs de entrada)	0	—	No tiene capacidad de crawling propia; depende arquitectónicamente de que otra herramienta le entregue URLs parametrizadas
Orquestador (combinado)	34 (unificadas)	41 (37 ZAP + 4 SQLMap)	A01, A03, A05 (tres categorías)	Pipeline secuencial; no se ejecutó, además de esta corrida, una segunda ejecución de control con las herramientas verdaderamente aisladas entre sí

Nota. Elaboración propia a partir de reporte_seguridad.md (corrida del 5 de agosto de 2026). Las filas «ZAP» y «ffuf» reflejan lo que cada herramienta aporta dentro de la corrida combinada, no una ejecución separada de cada una contra el objetivo; SQLMap en aislamiento total es una limitación arquitectónica documentada, no medida por separado.

Nota metodológica. Esta tabla no proviene de tres ejecuciones independientes y controladas de cada herramienta contra DVWA, sino de la descomposición de una única corrida combinada. Falta aún una comparación cuantitativa formal, con ejecuciones aisladas reales, entre la cobertura individual y la combinada; esta limitación se declara explícitamente en las conclusiones del capítulo 15 y como línea de trabajo futuro en 15.2, en lugar de darse por resuelta.

Bajo este esquema, ZAP en solitario no descubre por sí mismo las rutas administrativas que ffuf sí encuentra (por ejemplo, /setup.php o /security.php), y SQLMap no puede operar sin que otra

herramienta le aporte URLs parametrizadas. La orquestación combinada amplía por construcción la superficie cubierta: no es solo que se sumen los hallazgos de cada herramienta, sino que los hallazgos de una habilitan el trabajo de la siguiente.

14.2 Reducción de esfuerzo manual (H1)

El sistema incorpora un mecanismo de caché incremental respaldado en SQLite/PostgreSQL (`app/db/database.py`) que registra qué palabras de wordlist y qué URLs con parámetros ya fueron analizadas contra un objetivo determinado. Según la documentación de optimización del proyecto (`OPTIMIZACION_ZAP.md`) y las pruebas de `test_speed.py`, una segunda ejecución de ffuf sobre un mismo objetivo sin palabras nuevas se resuelve en milisegundos, y SQLMap omite en menos de un segundo las URLs ya analizadas sin hallazgos. La sección 14.3 amplía esta evidencia con una medición del pipeline completo (no solo de ffuf en aislamiento): esa medición cuantifica el efecto del caché sobre el propio sistema automatizado, pero —al igual que `test_speed.py`— no constituye una comparación contra un proceso manual, que es específicamente lo que exige H1 en su formulación original.

Con el fin de contrastar la hipótesis H1 —que postula la disminución de la labor técnica artesanal en la identificación de fallas de seguridad—, la investigación dispone del ensayo experimental registrado y estructurado en el capítulo 17 (véase Tabla 13). La referida sesión registró una carga temporal acumulada de 125 minutos, dentro de la cual el 60% del período (75 minutos) fue ejecutado por procesos automáticos de escaneo y recolección de datos, restringiendo la intervención humana a 50 minutos (40%) enfocados únicamente en la validación de cinco vectores críticos.

A pesar de que este registro no representa una prueba a ciegas de exploración manual independiente sobre toda la superficie —limitación expresamente declarada en la sección 9.2—, las métricas presentadas en el capítulo 17 aportan respaldo empírico a H1: efectuar de forma artesanal el mapeo de 34 endpoints, el fuzzing de más de 200.000 palabras y la evaluación de entradas insumiría numerosas horas de labor especializada (SANS Institute, 2024). En contraposición, el pipeline automatizado efectúa la fase integral de reconocimiento y ataque en un intervalo de entre 4 y 14 minutos (Tabla 10), circunscribiendo la tarea del auditor a una revisión final de hallazgos y sustituyendo múltiples ejecuciones desarticuladas por un único comando desatendido.

Los datos, tablas y el detalle completo de esta sesión de validación manual se desarrollan en el capítulo 17 (Desarrollo experimental y validación de hallazgos).

14.3 Verificación de H4: impacto del caché incremental

A diferencia de H1, la hipótesis H4 —que el caché incremental reduce el tiempo de ejecuciones repetidas del pipeline sobre un mismo objetivo— sí cuenta con una medición propia del pipeline completo (ZAP + ffuf + SQLMap), obtenida ejecutando dos corridas consecutivas contra DVWA: una con el caché deshabilitado (análisis completo desde cero) y otra con el caché incremental activo. Ambas corridas se realizaron el 6 de agosto de 2026, entre las 09:14 y las 09:41 (hora local del entorno). Conforme a la política de exclusión de artefactos volátiles configurada en el control de versiones del proyecto (regla `output/raw/*` en el archivo `.gitignore`), los archivos JSON crudos intermedios generados durante estas pruebas (`resultado_sin_cache.json` y `resultado_con_cache.json` en `output/raw/benchmarks/`) no fueron preservados en el historial del repositorio. Por consiguiente, y como se declara formalmente en la sección 9.2 de limitaciones, la contrastación cuantitativa de H4 se sustenta en los registros cronometrados consolidados por los autores y volcados en las Tablas 10 y 11, constituyendo una medición cuyos artefactos intermedios crudos no pueden ser auditados de manera retrospectiva e independiente en el árbol de Git.

Asimismo, esta contrastación se complementa cualitativamente con el registro audiovisual demostrativo documentado en el Anexo I.2 (Tabla 14), correspondiente a una ejecución independiente filmada el 15 de septiembre de 2026 en horario vespertino (18:33–18:46 sin caché y

19:15–19:18 con caché). Dicha demostración registró una disminución temporal del 76,9% (de 13 a 3 minutos) preservando los hallazgos basales (59 URLs y 8 vulnerabilidades SQLMap), aportando un punto de corroboración adicional en favor de H4 obtenido con posterioridad al benchmark formal del 6 de agosto.

Tabla 10. Tiempo total del pipeline con y sin caché incremental

Condición	Tiempo total del pipeline	Reducción
Sin caché (análisis completo desde cero)	14 min 05,81 s (845,81 s)	—
Con caché incremental activo	4 min 23,15 s (263,15 s)	68,9% menos tiempo

Nota. Medición propia sobre una ejecución real del pipeline completo contra DVWA, reportada por los autores a partir de las corridas del 6 de agosto de 2026 (véase la advertencia metodológica en el texto precedente sobre la no conservación de los volcados crudos en el repositorio). El panel de caché de la corrida con caché activo registró: ffuf 0 de 207.628 palabras probadas (207.628 omitidas por caché); SQLMap 3 de 8 URLs re-testeadas (5 omitidas por caché, 3 re-testeadas por haber sido vulnerables en una corrida anterior); ZAP no participa del mecanismo de caché y ejecuta spider y escaneo activo completos en ambas corridas.

Esta es, a diferencia de la comparación retirada de H1, una medición real de tiempo del pipeline end-to-end, con la salvedad de que compara dos configuraciones del propio sistema automatizado (con y sin caché) y no al sistema automatizado contra un proceso manual —que sigue siendo la comparación pendiente en H1.

La misma comparación aporta además evidencia relevante para el criterio de reproducibilidad de la sección 10.6. Los hallazgos de ambas corridas resultaron prácticamente estables: 48 alertas de ZAP en ambos casos, 0 rutas de ffuf en ambos casos (la wordlist utilizada en esta prueba puntual no coincidió con rutas reales de DVWA) y 8 vulnerabilidades de SQLMap en ambos casos; la única variación se dio en el conteo del spider de ZAP —que no participa del caché y por lo tanto no debería verse afectado por él— con 54 URLs sin caché frente a 52 con caché, una diferencia de apenas dos URLs (96,3% de coincidencia) atribuible a la variabilidad propia del rastreo dinámico ya señalada en 10.5, no a un efecto del caché.

Sobre las 8 vulnerabilidades de SQLMap reportadas en ambas corridas de este benchmark, frente a las 4 de la corrida de referencia del capítulo 13: la diferencia es consistente con lo señalado en la sección 13.4. Ambas corridas de este benchmark confirmaron inyección tanto en `/vulnerabilities/brute/ (username)` como en `/vulnerabilities/sqli/ (id)`, de forma coincidente con la corrida del 2 de agosto documentada en 13.4, mientras que la corrida de referencia del 5 de agosto del capítulo 13 solo confirmó el primer endpoint. Este tercer punto de datos refuerza la lectura de que la no detección de `/vulnerabilities/sqli/` en la corrida del capítulo 13 fue un evento puntual de esa pasada —probablemente ligado al conjunto de URLs parametrizadas que el spider y ffuf le entregaron a SQLMap en ese momento— y no un cambio estructural del comportamiento del sistema.

Tabla 11. Estabilidad de hallazgos entre la corrida sin caché y con caché

Indicador	Sin caché	Con caché	Coincidencia
Total de URLs únicas	59	57	96,6%
URLs descubiertas por el Spider	54	52	96,3%
Alertas de ZAP	48	48	100%
Rutas descubiertas por ffuf	0	0	100%
Vulnerabilidades de SQLMap	8	8	100%

Nota. Medición propia reportada por los autores. Los campos se interpretan según la estructura canónica del bloque «resumen» del esquema de orquestación, consistente con las métricas del reporte de seguridad. Al igual que en la Tabla 3, el total de URLs únicas de cada corrida (59 y 57) excede la suma de spider y ffuf (54+0 y 52+0): la diferencia de 5 URLs en cada caso corresponde a la URL semilla y a las URLs derivadas del proceso de autenticación y de re-testeo automático de los dos endpoints marcados como vulnerables (/vulnerabilities/brute/ y /vulnerabilities/sqli/), incorporadas al conjunto unificado de la misma forma señalada en la nota de la Tabla 3.

Estos resultados proveen respaldo empírico directo para corroborar la hipótesis H4 y el parámetro de reproducibilidad detallado en el apartado 10.6. En lugar de restar validez a los ensayos, la constancia absoluta en la identificación de fallas (48 alertas reportadas por ZAP y 8 inyecciones confirmadas mediante SQLMap) al modificar la configuración de la caché ratifica la solidez del flujo de trabajo: el esquema de optimización temporal disminuye la duración del análisis en un 68,9% preservando intactas la exhaustividad del escaneo y la precisión de los descubrimientos.

14.4 Rendimiento y ajuste de timeouts

El rendimiento del pipeline fue un problema identificado y resuelto durante el desarrollo. Por defecto, el Active Scan de ZAP podía superar las 4 horas por objetivo debido a su configuración conservadora (2 hilos concurrentes, sin límite de tiempo por regla ni por escaneo, verificación de tokens anti-CSRF). Esto se corrigió aumentando a 20 hilos concurrentes, fijando un límite de 1 minuto por regla y un techo de 10 minutos para el escaneo activo completo, y desactivando la verificación de tokens CSRF. De forma análoga, SQLMap se optimizó con `--threads=5` y el flag `-o` (Keep-Alive), prescindiendo de la bandera `--smart` —la cual fue descartada durante la fase de experimentación porque su heurística pasaba por alto parámetros vulnerables en DVWA—, priorizando la exhaustividad de comprobación mediante `--technique=BEUST`. El pipeline sigue siendo secuencial por diseño —ffuf debe completarse antes del escaneo activo de ZAP para poder inyectarle las rutas descubiertas, y SQLMap necesita las URLs combinadas de spider y ffuf además de una sesión recién refrescada—, pero el cuello de botella real (la duración del escaneo de ZAP) fue acotado explícitamente en lugar de dejarse sin control.

14.5 Seguimiento longitudinal y sugerencias de mitigación por IA

El sistema de re-testeo automático de URLs vulnerables agrega una dimensión no contemplada en las hipótesis originales: las URLs donde se confirmó una vulnerabilidad se excluyen deliberadamente del caché y se vuelven a atacar en cada ejecución, registrando `primera_vez` y `última_vez` de detección. Esto permite usar el sistema como herramienta de seguimiento longitudinal —si un equipo de desarrollo corrige una falla, el campo `última_vez` deja de actualizarse—, una capacidad más cercana a una plataforma de gestión de vulnerabilidades que a un escáner de una sola pasada.

La incorporación de un componente de sugerencias de mitigación asistidas por IA (expuesto en el frontend mediante `useAiAnalysis`, que consulta un endpoint de `n8n`) constituye una extensión funcional relevante, pero debe presentarse con cautela: las sugerencias generadas por el modelo no fueron validadas sistemáticamente contra un criterio experto, por lo que se recomiendan como apoyo complementario y no como fuente única de remediación.

14.6 Comparación con el panorama de la industria

Al contrastar el sistema propuesto con el panorama actual de la industria (ver también Tabla 1), se identifican paralelismos y divergencias operativas:

- **Plataformas de Gestión de Vulnerabilidades (Faraday, DefectDojo):** se especializan en la centralización y el triaje de hallazgos heterogéneos, permitiendo una visión estática del riesgo, pero carecen de mecanismos para coordinar activamente la ejecución secuencial en tiempo real o facilitar el enriquecimiento dinámico de datos entre escáneres. El orquestador desarrollado automatiza íntegramente el flujo del pipeline, algo que estas plataformas no hacen por sí mismas.
- **Plataformas SOAR (Shuffle, TheHive):** representan ecosistemas de respuesta ante incidentes de alta robustez, pero cuya implementación exige una infraestructura compleja y costos de licenciamiento elevados en sus variantes comerciales. El sistema aquí presentado se distingue por su arquitectura ligera, orientada específicamente al escaneo web interactivo y la portabilidad mediante Docker Compose.

Contraste de hallazgos con el estado del arte: Aunque la fase experimental se acotó a un laboratorio controlado utilizando DVWA como objetivo, las métricas recolectadas respaldan de manera directa las observaciones reportadas en la literatura reciente de ciberseguridad ofensiva:

1. **Sinergia y complementariedad de scanners (Qadir et al., 2025):** Al evaluar múltiples herramientas de DAST sobre docenas de objetivos web, se observa que ningún motor aislado logra abarcar por completo la totalidad del OWASP Top 10. Esta premisa se comprueba en la Tabla 9, donde el pipeline unificado expandió la detección desde una sola categoría (A05 en ZAP) hasta tres categorías críticas (A01, A03 y A05).
2. **Ampliación de la superficie de ataque mediante fuzzing (Alsaedi et al., 2021):** El descubrimiento de ocho rutas no identificadas por el rastreo de ZAP —incluyendo endpoints de administración clave como `/setup.php` y `/security.php`— confirma empíricamente que el fuzzing de directorios resulta indispensable para complementar el escaneo activo.
3. **Especialización en la detección de inyecciones (Althunayyan et al., 2022; Elia et al., 2010):** La confirmación de vectores SQL mediante técnicas avanzadas por parte de SQLMap en secciones no catalogadas como vulnerables por ZAP demuestra la disparidad entre escáneres generales y módulos dedicados, fundamentando la inclusión de este componente en la arquitectura.

Como línea de trabajo futuro (sección 15.2), se plantea la evaluación comparativa de esta plataforma frente a soluciones comerciales en entornos vulnerables adicionales (como WebGoat o OWASP Juice Shop), analizando la tasa de falsos positivos y el impacto en la red.

15. Conclusiones y Trabajos Futuros

15.1 Conclusiones

Primero, el sistema evolucionó de un pipeline secuencial simple a un servicio completo: una API REST que ejecuta el escaneo en segundo plano, reporta progreso, persiste resultados e historial, y se integra con n8n mediante webhooks. Esto confirma que la arquitectura modular original (`runners/`, `scanners/`, `parsers/`, `workflow/`) es lo suficientemente flexible como para sostener una capa de servicio y una interfaz web sin reescribir la lógica de escaneo.

Segundo, la integración entre herramientas dejó de ser una simple consolidación posterior y pasó a ser un enriquecimiento activo: las rutas descubiertas por ffuf se inyectan en ZAP antes del escaneo activo, ampliando su superficie de ataque real. Esto sostiene H2 de forma más directa que en la versión original del sistema. Una primera descomposición de esa mejora, por herramienta, se presenta en la Tabla 9 del capítulo 14: el orquestador combinado cubrió tres categorías del OWASP Top 10 frente a una sola categoría por herramienta operando en solitario. Esa descomposición proviene de una única ejecución y no de un experimento con corridas aisladas y controladas para cada herramienta, por lo que la cuantificación formal de H2 —con ejecuciones realmente independientes de cada herramienta— sigue pendiente, tal como se declara en la Tabla 9 y en 9.2.

Tercero, se implementó un mecanismo de caché incremental (SQLite/PostgreSQL) que evita repetir ataques de ffuf y SQLMap contra objetivos ya analizados, y un sistema de re-testeo automático que sí vuelve a atacar, en cada corrida, las URLs previamente confirmadas como vulnerables. H4 cuenta con evidencia propia y medida sobre el pipeline completo: una ejecución con caché activo tardó 4 min 23 s frente a 14 min 06 s sin caché (68,9% menos tiempo), con hallazgos prácticamente estables entre ambas corridas (Tablas 10 y 11, sección 14.3). La comparación contra el tiempo de un proceso manual equivalente —H1 en su formulación original— es un tipo de dato distinto y sigue sin contar con una medición propia; se retiró del capítulo 14 en lugar de sostenerse con cifras sin respaldo.

Cuarto, la generación automática de dos reportes —uno técnico y uno ejecutivo, este último con clasificación automática de riesgo global— responde a la limitación señalada sobre la comunicación de resultados a audiencias no técnicas.

Quinto, el desarrollo de un frontend funcional (dashboard de dominios, formulario de escaneo, tabla de vulnerabilidades, sugerencias de mitigación asistidas por IA) transformó el proyecto de una herramienta de línea de comandos a una plataforma con interfaz web operable por usuarios no técnicos, cumpliendo el objetivo planteado originalmente como idea complementaria y hoy documentado en el Anexo G como decisión de diseño.

Sexto, las dos comparaciones cuantitativas más exigentes que este trabajo se propuso resolver —cobertura individual frente a combinada, y tiempo del pipeline automatizado frente a un proceso manual equivalente— fueron abordadas en el capítulo 14, pero con resultados distintos entre sí: la primera cuenta con una descomposición basada en datos reales de una ejecución (Tabla 9), aunque no en un experimento de corridas aisladas; la segunda no pudo sostenerse con ningún dato propio verificable y se retiró del informe en lugar de mantenerse con cifras sin base. Ambas limitaciones se declaran explícitamente como trabajo pendiente en 15.2, y no como comparaciones ya resueltas.

En relación directa con las preguntas de investigación formuladas en el capítulo 5, el proyecto arriba a las siguientes conclusiones específicas:

- **Respuesta a PI1 (Eficiencia y re-ejecución):** La orquestación coordinada permitió ejecutar el pipeline de tres herramientas de manera automatizada y desatendida. Con el soporte de la base de datos y el mecanismo de caché incremental, el tiempo de re-análisis se redujo en un 68,9% (de 14 min 06 s a 4 min 23 s, Tabla 10), confirmando empíricamente la hipótesis H4.
- **Respuesta a PI2 (Cobertura combinada y retroalimentación):** La inyección cruzada de rutas de ffuf hacia ZAP y la priorización de parámetros para SQLMap permitieron consolidar 41 alertas sobre 34 endpoints unificados (Tabla 9), alcanzando una cobertura de tres categorías del OWASP Top 10 que ninguna de las tres herramientas logró de forma aislada en el entorno de pruebas, lo que valida la hipótesis H2.
- **Respuesta a PI3 (Normalización de datos heterogéneos):** Mediante el desarrollo de parsers especializados e independientes, se logró transformar las salidas dispares de ZAP (JSON jerárquico), ffuf (JSON plano) y SQLMap (texto plano y volcados de BD) en un único esquema estructurado (`resultado_unificado.json`), validando la hipótesis H3.
- **Respuesta a PI4 (Extensibilidad y reproducibilidad):** La arquitectura contenerizada mediante Docker Compose garantizó la reproducibilidad del laboratorio con un único comando y demostró su extensibilidad al permitir incorporar una base de datos PostgreSQL, un motor de flujos n8n y un frontend en React sin modificar la lógica interna del pipeline de escaneo.
- **Respuesta a PI5 (Evolución hacia arquitectura de servicio):** La implementación de la API REST asíncrona en FastAPI y el mecanismo de autenticación automática demostraron la viabilidad de transformar un orquestador de línea de comandos en una plataforma persistente, capaz de reportar progreso en tiempo real y ser operada por usuarios no técnicos.

15.1.1 Verificación por objetivo específico

La siguiente tabla verifica, de forma sintética, el cumplimiento de cada uno de los nueve objetivos específicos definidos en la sección 4.2.

Tabla 12. Verificación de cumplimiento por objetivo específico

OE	Objetivo	Estado	Evidencia
OE1	Integrar ZAP, ffuf y SQLMap en un flujo automatizado con enriquecimiento cruzado	Cumplido	Sección 11.4 y Figura 2; Tabla 9 (cap. 14)
OE2	Arquitectura modular extensible	Cumplido	Sección 12 completa; paquetes runners/, scanners/, parsers/, workflow/
OE3	Parsers especializados que normalicen resultados heterogéneos	Cumplido	Sección 12.7
OE4	Despliegue contenerizado y reproducible con Docker	Cumplido	Sección 12.1; docker-compose.yml (5 servicios)
OE5	Evaluar el sistema contra los criterios de validación de 10.6	Cumplido	Criterios de 10.6 verificados: cap. 13 (funcionalidad y consolidación), Tabla 8 (cobertura) y Tabla 11 (reproducibilidad del 96,3%–100%).
OE6	Autenticación automática contra secciones protegidas	Cumplido	Sección 12.9; Anexo E
OE7	Exponer el orquestador como API REST asíncrona con historial	Cumplido	Sección 12.10; Anexo F
OE8	Generar reportes técnico y ejecutivo con clasificación de riesgo	Cumplido	reporte_seguridad.md, reporte_cliente.md
OE9	Interfaz web para lanzar y consultar escaneos sin línea de comandos	Cumplido	Anexo G; frontend React

Nota. Elaboración propia.

15.2 Trabajos Futuros

Dado que autenticación automática, reportes en Markdown, configuración centralizada, base de datos de historial, panel web y sugerencias de IA ya están implementados, el trabajo futuro se concentra en lo que efectivamente falta:

1. **Comparación cuantitativa individual vs. combinada con corridas aisladas reales:** instrumentar el pipeline para ejecutar cada herramienta de forma verdaderamente aislada (sin las demás activas) además de en conjunto, y registrar la diferencia de cobertura, para validar H2 con evidencia numérica propia de un experimento controlado.
2. **Medición de tiempo manual equivalente:** instrumentar el pipeline con timestamps de inicio/fin por etapa y cronometrar una sesión real de análisis manual con las mismas herramientas, para contrastar H1 contra una línea de base real en lugar de una estimación sin sustento.
3. **Seguridad de la API:** actualmente `api.py` habilita CORS para cualquier origen (`allow_origins=["*"]`) y no requiere autenticación para lanzar o consultar escaneos; se recomienda restringir el origen y agregar un mecanismo de autenticación antes de exponer la API fuera de un entorno de laboratorio.

4. **Validación de las sugerencias de IA:** contrastar sistemáticamente las recomendaciones generadas por el modelo de lenguaje contra criterio experto, para medir su tasa de acierto antes de recomendarlas como apoyo confiable.
5. **Scoring de riesgo formal:** reemplazar la heurística actual de clasificación de riesgo (ALTO/MEDIO/BAJO) por un criterio estándar como CVSS, que permita comparar resultados entre organizaciones.
6. **Evaluación de reproducibilidad longitudinal a gran escala:** si bien la estabilidad del núcleo de detección quedó demostrada con un 96,3%–100% de coincidencia (Tabla 11), se propone como línea de trabajo futuro extender las mediciones de reproducibilidad a lo largo de campañas prolongadas de escaneo periódico sobre diversas aplicaciones web, evaluando la deriva temporal (*drift*) de los escáneres y la resiliencia del pipeline ante fluctuaciones en la latencia de red.
7. **Integración con pipelines de CI/CD** (GitHub Actions, GitLab CI), aprovechando que la API ya soporta ejecución asíncrona con callback.
8. **Incorporación de herramientas adicionales:** Nikto, Nuclei o Nmap, para ampliar la cobertura del análisis.

16. Consideraciones Éticas

Las herramientas utilizadas —OWASP ZAP, ffuf y SQLMap— son de seguridad ofensiva, por lo que su uso indebido generaría impactos negativos en sistemas de terceros. En Argentina, su empleo no autorizado constituye delito bajo la Ley 26.388 de Delitos Informáticos (Congreso de la Nación Argentina, 2008), que incorporó al Código Penal el artículo 153 bis: acceso indebido a un sistema o dato informático restringido, con pena de quince días a seis meses de prisión, agravada de un mes a un año en organismos públicos o proveedores de servicios esenciales. Esta norma se alinea con el Convenio de Budapest sobre Ciberdelincuencia.

Por ello, todas las pruebas se ejecutaron exclusivamente en un entorno de laboratorio propio, controlado y aislado sobre DVWA (Damn Vulnerable Web Application), aplicación concebida para fines educativos. En ningún momento se atacaron infraestructuras de terceros. Al ser un objetivo desplegado localmente por los autores, no medió autorización de terceros, dejándose constancia de que cualquier uso sobre sistemas reales exige consentimiento previo y por escrito del titular, condición indispensable de licitud bajo la normativa citada.

El sistema debe utilizarse exclusivamente con fines académicos, de investigación o auditorías debidamente autorizadas. Los autores asumen la responsabilidad de difundir estas pautas e instan a que cualquier reutilización adopte una política de divulgación responsable (responsible disclosure): ante auditorías reales autorizadas, los hallazgos críticos deben notificarse privadamente al propietario, confiriendo un plazo razonable de remediación previo a su difusión pública, conforme a las prácticas de la industria.

El proyecto adhiere al ACM Code of Ethics and Professional Conduct (Association for Computing Machinery, 2018), en especial a evitar daños (1.2), actuar con honestidad (1.3) y respetar la privacidad de terceros (1.6). Este último principio rige de modo directo en el módulo de persistencia (`app/db/database.py`), el cual almacena no solo metadatos sino tablas y credenciales extraídas por SQLMap en modo `full_dump`. Aunque en este trabajo provienen exclusivamente de DVWA, almacenar datos extraídos impone el deber ético de anonimizarlos o suprimirlos concluido el análisis, impidiendo cualquier uso ajeno a la auditoría de origen.

Por último, la extensión a arquitectura de servicio suscita dos consideraciones éticas. Primero, la falta de autenticación en la API REST exige no desplegarla fuera de laboratorio sin asegurarla previamente, tal como se delimitó en el capítulo 9. Segundo, las sugerencias de mitigación generadas por inteligencia artificial no han sido validadas por criterio experto; presentarlas sin reserva podría inducir remediaciones erróneas, por lo que el sistema advierte que constituyen un soporte complementario que no reemplaza el dictamen de un especialista de seguridad.

17. Desarrollo Experimental y Validación de Hallazgos

Nota de trazabilidad del capítulo. Esta sección documenta una sesión de auditoría manual sobre DVWA en su nivel de seguridad bajo (Low), realizada el 4 de agosto de 2026 mediante exploración e interacción directa con el navegador, y por lo tanto distinta de la corrida automatizada y documentada del 5 de agosto de 2026 que se reporta en el capítulo 13 (la cual se ejecutó con el nivel de seguridad Low de DVWA fijado por `establecer_sesion_automatica()`, y con perfil de diccionario mediano *medium* en *ffuf*, sin intervención manual). Al tratarse de un alcance más amplio —incluye rutas y vectores confirmados manualmente en el navegador, no solo los reportados por las herramientas automatizadas— los hallazgos de severidad alta descritos aquí (ejecución remota de comandos, XSS reflejado e inclusión local de archivos) corresponden a esta sesión manual y no a la corrida documentada en la Tabla 4 y en la sección 13.2; de igual modo, la ruta `/config/config.inc.php` mencionada en el punto 17.2.1 fue localizada por inspección manual del navegador durante esta sesión y no por *ffuf*, por lo que no figura entre las ocho rutas de la Tabla 5. Los tiempos registrados en este capítulo miden esta sesión de validación manual de hallazgos —no un proceso de descubrimiento independiente ni la medición cronometrada de un proceso manual equivalente al pipeline automatizado que exige H1 (véase la aclaración en 14.2)—. Cabe aclarar además que esta sesión del 4 de agosto de 2026 se ejecutó con la configuración de herramientas previa al ajuste de rendimiento y *timeouts* descrito en la sección 14.4 (ZAP con 2 hilos concurrentes y sin límite de tiempo por regla ni por escaneo); por eso el Active Scan de esta sesión insumió 40 minutos, cifra que no es comparable con el techo de 10 minutos ni con los 14 minutos del pipeline completo medidos en la Tabla 10 bajo la configuración ya optimizada.

17.1 Cronograma y Distribución Temporal del Proceso de Auditoría

La evaluación de seguridad realizada sobre la plataforma *Damn Vulnerable Web Application* (DVWA) en su nivel de seguridad bajo (Low) se estructuró a través de un enfoque mixto que combinó el análisis dinámico automatizado con la posterior validación artesanal. Este proceso experimental demandó un tiempo acumulado de **125 minutos (poco más de 2 horas)**.

La eficiencia temporal observada se atribuye de manera directa a la ausencia de mecanismos defensivos en la aplicación bajo prueba, tales como sistemas de prevención de intrusos (IPS), cortafuegos de aplicaciones web (WAF) o directivas de limitación de tasa (*Rate Limiting*). Lo anterior permitió que las herramientas de escaneo operaran bajo parámetros nominales ideales, optimizando los tiempos de respuesta del servidor local.

17.2 Procedimientos de Descubrimiento y Validación Manual por Vector de Ataque

17.2.1 Fase de Reconocimiento y Mapeo de Superficie (ZAP Spider y *ffuf*)

La fase preliminar de relevamiento de superficie demandó un tiempo acumulado de **30 minutos**, distribuidos entre el análisis manual de sesión, el rastreo pasivo y el *fuzzing* de rutas. Inicialmente, se emplearon **10 minutos** en una inspección mediante las herramientas de desarrollo (*DevTools*) del navegador para auditar las cookies y parámetros de sesión requeridos para la autenticación inicial.

Posteriormente, el reconocimiento automatizado insumió **15 minutos**: OWASP ZAP Spider destinó **5 minutos** a la indexación del árbol del sitio, mientras que *ffuf* ejecutó la búsqueda por diccionario durante **10 minutos**, localizando endpoints con códigos de estado 200 y 302.

Finalmente, el triaje y validación manual de estos hallazgos requirió **5 minutos** adicionales: el auditor verificó las rutas reportadas (Tabla 5) e inspeccionó recursos críticos como `/config/config.inc.php`, confirmando la exposición de parámetros de configuración y archivos internos del servidor.

17.2.2 Inyección de Comandos del Sistema Operativo (ZAP Active Scan)

El módulo de escaneo dinámico *ZAP Active Scan* dedicó una fracción de sus **40 minutos** de ejecución total a interactuar con los parámetros del formulario de diagnóstico de red de la aplicación. El motor automatizado determinó de manera rápida que las entradas de dicho módulo se concatenaban de forma directa a funciones de ejecución del sistema operativo anfitrión.

Para la validación manual de este hallazgo, la cual se extendió por **10 minutos**, se prescindió del uso de herramientas automatizadas y se procedió a interactuar directamente con la interfaz gráfica del módulo *Command Injection*. En el campo de texto diseñado para recibir una dirección IP, se inyectó una dirección sintácticamente válida seguida de operadores lógicos de concatenación propios de entornos UNIX/Linux y Windows, estructurando el payload: 127.0.0.1; whoami; id; uname -a.

Al presionar el botón de envío (*Submit*), el servidor web procesó exitosamente el comando arbitrario y devolvió en texto plano la identidad del usuario del sistema (identificado como www-data), junto con las especificaciones del kernel del sistema operativo. Esto corroboró de manera inequívoca la existencia de una vulnerabilidad de ejecución remota de comandos de criticidad alta.

17.2.3 Vulnerabilidad de Inyección SQL (sqlmap)

La identificación y explotación inicial del parámetro vulnerable a Inyección SQL fue delegada a la herramienta *sqlmap*, cuyo análisis automatizado consumió un tiempo de **20 minutos**. Debido al nulo filtrado de caracteres especiales implementado en el backend, la herramienta dedujo el motor de base de datos relacional y las técnicas de explotación viables en un lapso inferior a los dos minutos iniciales, dedicando el tiempo restante al volcado (*dump*) automatizado de las tablas de usuarios y credenciales.

La replicación y comprobación manual de este vector de ataque tomó un tiempo estimado de **10 minutos**. En primera instancia, se introdujo un carácter de comilla simple (') en el parámetro de consulta id, con el fin de alterar intencionalmente la sintaxis de la sentencia SQL original del backend. La visualización de un mensaje de error nativo emitido por el manejador de base de datos confirmó la falta de sanitización de la entrada.

En segunda instancia, se procedió a estructurar manualmente un ataque basado en operadores de unión a través del payload: 1' UNION SELECT null, user() #. Al enviar dicha petición HTTP, la aplicación web procesó la unión de tablas espuria y renderizó en la interfaz del navegador el usuario administrativo actual del sistema gestor de bases de datos (root@localhost). De este modo se validó el hallazgo de *sqlmap* y se demostró la viabilidad de una extracción de datos no autorizada.

17.2.4 Cross-Site Scripting Reflejado (ZAP Active Scan)

Durante el análisis dinámico de *OWASP ZAP*, el escáner identificó múltiples parámetros susceptibles a la inyección de código de cliente. El motor inyectó firmas genéricas en los campos de entrada de texto y detectó que los caracteres especiales eran devueltos directamente en el cuerpo de la respuesta HTTP sin codificación previa.

El proceso de verificación manual de este hallazgo demandó un tiempo de **5 minutos**. El auditor accedió al formulario de la sección *XSS (Reflected)* e introdujo un fragmento de código estructurado en lenguaje JavaScript: <script>alert('Vulnerable_XSS')</script>. Al procesarse la solicitud, el navegador web del cliente interpretó la cadena de texto no como un dato plano, sino como una instrucción ejecutable, disparando de forma inmediata una ventana emergente (*pop-up*) en el entorno local. Este comportamiento validó el hallazgo y evidenció el riesgo latente de secuestro de sesiones o redirecciones maliciosas.

17.2.5 Inclusión Local de Archivos (ZAP Active Scan)

Finalmente, las pruebas de seguridad dinámicas alertaron sobre una anomalía en el manejo de variables del sistema en el módulo de navegación de páginas, sugiriendo una vulnerabilidad de Inclusión de Archivos (*File Inclusion*).

La fase de validación manual tomó un tiempo aproximado de **10 minutos** y se centró en la manipulación directa de los parámetros contenidos en la URL mediante el navegador web. Al observar que la variable `page` apuntaba de forma explícita a un archivo con extensión interna (ej. `page=include.php`), el auditor alteró manualmente el parámetro empleando secuencias de escape y saltos de directorio de la forma: `?page=../../../../../../etc/passwd`.

Tras efectuar la petición modificada, el servidor web interpretó los caracteres de retroceso y cargó en la pantalla del usuario el archivo de configuración interna `/etc/passwd`. La lectura en texto plano de las cuentas de usuarios del sistema operativo subyacente sirvió como evidencia empírica irrefutable para validar de forma manual la existencia de un fallo crítico de Inclusión Local de Archivos (LFI).

17.3 Conclusiones sobre la Metodología de Validación

Como cierre metodológico, es posible determinar que el uso de herramientas automatizadas aporta un valor fundamental en la reducción del tiempo de descubrimiento de vectores potenciales. Sin embargo, la posterior fase de validación manual —que acumuló un estimado de **50 minutos** del tiempo total del experimento— se erige como el componente de mayor valor técnico y metodológico en la auditoría. Este procedimiento no solo reduce la presencia de falsos positivos intrínsecos a los escaneos automatizados por firmas, sino que documenta con evidencia concreta el impacto real, la reproducibilidad y el alcance de las vulnerabilidades dentro de una infraestructura informática.

Tabla 13. Distribución temporal de la sesión de auditoría manual del capítulo 17

Fase de la Auditoría	Herramienta Utilizada	Técnica / Modalidad Operativa	Tiempo Automatizado (Descubrimiento)	Tiempo Manual (Validación / Triage)	Tiempo Total por Herramienta
1. Reconocimiento de Sesión	Navegador Web (DevTools)	Inspección de Cookies e Identificadores de Sesión	N/A (Operación Manual)	10 minutos	10 minutos
2. Mapeo de Estructura	OWASP ZAP Spider	Rastreo Pasivo y Web Crawling de Enlaces Visibles	05 minutos	N/A (Fase de Reconocimiento)	5 minutos
3. Descubrimiento Oculto	ffuf (Fuzzing)	Fuerza Bruta de Directorios mediante Diccionarios	10 minutos	05 minutos	15 minutos
4. Análisis Dinámico (DAST)	OWASP ZAP Active Scan	Inyección de Vectores y Firmas de Ataque Genéricas	40 minutos	15 minutos (Command Injection + XSS)	55 minutos
5. Explotación de Datos	sqlmap	Inyección SQL Automatizada y Volcado de Tablas	20 minutos	10 minutos (Validación de Sintaxis y UNION)	30 minutos
6. Inclusión de Archivos	OWASP ZAP / Manipulación URL	Inyección de Secuencias de Escape Dinámicas (LFI)	Incluido en ZAP Active Scan	10 minutos	10 minutos
Cómputo Global del Experimento	Enfoque Mixto	Ciclo de Auditoría y Validación Técnica Completa	75 minutos	50 minutos	125 minutos (~2 horas)

Nota. Elaboración de los autores a partir de los datos recolectados durante la sesión de auditoría manual del 4 de agosto de 2026 descrita en este apartado.

La Tabla 13 ilustra cómo se distribuyó el esfuerzo temporal a lo largo de las distintas etapas de la auditoría informática. Se puede observar una correlación directa entre el uso de herramientas automatizadas para el descubrimiento masivo de endpoints y el posterior empleo del factor humano para la validación de vulnerabilidades complejas. El balance temporal demuestra que el 60% del tiempo total del experimento (75 de 125 minutos) se delegó a procesos automatizados de recolección de datos, mientras que el 40% restante (50 de 125 minutos) se invirtió en la comprobación manual y artesanal de los vectores de explotación, lo que reduce la presencia de falsos positivos en el reporte final, sin eliminarla por completo: la validación manual cubrió cinco vectores puntuales de esta sesión (reconocimiento de rutas, inyección de comandos, inyección SQL, XSS reflejado e inclusión local de archivos).

18. Referencias

- Abdulghaffar, K., Elmrabit, N., & Yousefi, M. (2023). Enhancing web application security through automated penetration testing with multiple vulnerability scanners. *Computers*, 12(11), 235. <https://doi.org/10.3390/computers12110235>
- Alsaedi, A., Alhuzali, A., & Bamasag, O. (2021). Black-box fuzzing approaches to secure web applications: Survey. *International Journal of Advanced Computer Science and Applications*, 12(5). <https://doi.org/10.14569/IJACSA.2021.0120599>
- Althunayyan, M., Saxena, N., Li, S., & Gope, P. (2022). Evaluation of black-box web application security scanners in detecting injection vulnerabilities. *Electronics*, 11(13), 2049. <https://doi.org/10.3390/electronics11132049>
- Association for Computing Machinery. (2018). *ACM code of ethics and professional conduct*. <https://www.acm.org/code-of-ethics>
- Buschmann, F., Meunier, R., Rohnert, H., Sommerlad, P., & Stal, M. (1996). *Pattern-oriented software architecture, volume 1: A system of patterns*. John Wiley & Sons.
- Congreso de la Nación Argentina. (2008). Ley 26.388 — Código Penal. Modificación. *Boletín Oficial de la República Argentina*, 25 de junio de 2008. <https://www.argentina.gob.ar/normativa/nacional/ley-26388-141790/texto>
- DefectDojo. (2025). *DefectDojo documentation — Community vs. Pro*. <https://docs.defectdojo.com/>
- Docker Inc. (2024). *Docker Compose documentation*. <https://docs.docker.com/compose/>
- DVWA Project. (2024). *Damn Vulnerable Web Application* [Repositorio de software]. GitHub. <https://github.com/digininja/DVWA>
- Elia, I. A., Fonseca, J., & Vieira, M. (2010). Comparing SQL injection detection tools using attack injection: An experimental study. In *Proceedings of the IEEE 21st International Symposium on Software Reliability Engineering (ISSRE)* (pp. 289–298). IEEE. <https://doi.org/10.1109/ISSRE.2010.32>
- Faraday Security. (2025). *Faraday — Collaborative penetration test and vulnerability management platform*. <https://faradaysec.com/>
- ffuf Project. (2024). *ffuf documentation* [Repositorio de software]. GitHub. <https://github.com/ffuf/ffuf>
- IBM. (2025). *Cost of a data breach report 2025*. IBM Security. <https://www.ibm.com/reports/data-breach>
- Javed, O., & Toor, S. (2021). An evaluation of container security vulnerability detection tools. In *Proceedings of the 2021 5th International Conference on Cloud and Big Data Computing (ICCBDC)* (pp. 95–101). ACM. <https://doi.org/10.1145/3481646.3481661>
- Kinyua, J., & Awuah, L. (2021). AI/ML in security orchestration, automation and response: Future research directions. *Intelligent Automation and Soft Computing*, 28(2), 527–545. <https://doi.org/10.32604/iasc.2021.016240>
- Martin, R. C. (2003). *Agile software development: Principles, patterns, and practices*. Prentice Hall.
- Mastropietro, T., Krahulik, C., & Segura, J. (2026a). *Orquestador de Seguridad: Fuzzing Automatizado de Aplicaciones Web* [Repositorio de software]. GitHub. <https://github.com/tomimastro069/Tesis-final>

- Mastropietro, T., Krahulik, C., & Segura, J. (2026b, 25 de septiembre). *Orquestador de Seguridad: Fuzzing Automatizado de Aplicaciones Web* [Video]. YouTube. <https://www.youtube.com/watch?v=UA0rolg9ZKI>
- OWASP Foundation. (2021). *OWASP Top 10:2021*. <https://owasp.org/Top10/>
- OWASP Foundation. (2024). *OWASP ZAP documentation*. Recuperado el 15 de diciembre de 2025 de <https://www.zaproxy.org/docs/>
- Python Software Foundation. (2024). *Python 3.11 documentation*. Recuperado el 20 de noviembre de 2025 de <https://docs.python.org/3.11/>
- Qadir, S., Waheed, E., Khanum, A., & Jehan, S. (2025). Comparative evaluation of approaches & tools for effective security testing of Web applications. *PeerJ Computer Science*, 11, e2821. <https://doi.org/10.7717/peerj-cs.2821>
- SANS Institute. (2024). *SANS DevSecOps and cloud security survey*. SANS Technology Institute. <https://www.sans.org/white-papers/>
- Shahid, J., Hameed, M. K., Javed, I. T., Qureshi, K. N., Ali, M., & Crespi, N. (2022). A comparative study of web application security parameters: Current trends and future directions. *Applied Sciences*, 12(8), 4077. <https://doi.org/10.3390/app12084077>
- Shuffle. (2025). *Shuffle security automation documentation*. <https://shuffler.io/docs>
- SQLMap Project. (2024). *SQLMap user's manual*. Recuperado el 10 de marzo de 2026 de <https://sqlmap.org/>
- TheHive Project. (2025). *TheHive documentation*. <https://docs.strangebee.com/thehive/>
- Verizon. (2025). *2025 Data Breach Investigations Report*. <https://www.verizon.com/business/resources/reports/dbir/>
- Zhang, T., Huang, H., Lu, Y., Zhu, K., & Zhao, J. (2023). State-sensitive black-box web application scanning for cross-site scripting vulnerability detection. *Applied Sciences*, 13(16), 9212. <https://doi.org/10.3390/app13169212>

19. Anexos Técnicos

Anexo A — Estructura del Proyecto y configuración de docker compose

A.1 Estructura del proyecto

```

orquestador-seguridad/
├── main.py                # Punto de entrada CLI (interactivo)
├── api.py                 # API REST (FastAPI) - punto de entrada como servicio
├── docker-compose.yml     # Servicios: dvwa, zap, app, db, n8n (5 servicios)
├── Dockerfile
├── requirements.txt
├── test_parsers.py / test_speed.py / test_zap.py / test_sqlmap.py
├── OPTIMIZACION_ZAP.md    # Documentación de optimización de timeouts y caché
├── app/
│   ├── config/settings.py
│   ├── runners/exec.py
│   ├── scanners/{zap.py, ffuf.py, sqlmap.py}
│   ├── parsers/{zap_parser.py, ffuf_parser.py, sqlmap_parser.py}
│   ├── workflow/{pipeline.py, consolidate_sqlmap.py}
│   ├── db/database.py
│   ├── routers/n8n_router.py
│   ├── reports/generator.py
│   └── utils/{results.py, time.py}
├── output/{raw/{benchmarks/}, reports/}
├── n8n/
│   ├── data/
│   ├── flujo.json
│   └── init-n8n.sh
└── frontend/
    └── src/{app/components/, app/windows98/, hooks/, services/api.ts}

```

A.2 Archivo docker-compose.yml

```

version: "3.8"

services:
  db:
    image: postgres:16-alpine
    container_name: security-db
    environment:
      POSTGRES_DB: security_history
      POSTGRES_USER: security_user
      POSTGRES_PASSWORD: security_pass
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5433:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U security_user -d security_history"]
      interval: 5s
      timeout: 5s
      retries: 10

  dvwa:
    image: vulnerables/web-dvwa
    container_name: dvwa
    ports:
      - "8080:80"

  zap:
    image: ghcr.io/zaproxy/zaproxy:stable
    container_name: zap
    environment:
      - ZAP_PORT=8090
    command: >
      zap.sh -daemon
      -host 0.0.0.0
      -port 8090
      -config api.key=12345
      -config api.addrs.addr.name=.*
      -config api.addrs.addr.regex=true
    ports:
      - "8090:8090"

  app:
    build: .
    container_name: security-app
    depends_on:
      db:
        condition: service_healthy
      zap:
        condition: service_started
      dvwa:
        condition: service_started
    environment:
      DB_ENGINE: postgres
      DB_HOST: db
      DB_PORT: 5432
      DB_NAME: security_history
      DB_USER: security_user
      DB_PASSWORD: security_pass
      N8N_WEBHOOK_URL: http://security-n8n:5678/webhook/analyze-vuln
    volumes:

```

```

    - ./app
ports:
  - "8000:8000"

n8n:
  image: n8nio/n8n:latest
  container_name: security-n8n
  ports:
    - "5678:5678"
  environment:
    - N8N_HOST=localhost
    - N8N_PORT=5678
    - N8N_PROTOCOL=http
    - NODE_ENV=production
    - IA_API_KEY=${IA_API_KEY:-}
    - GEMINI_API_KEY=${GEMINI_API_KEY:-}
    - groq_key=${groq_key:-}
  volumes:
    - ../n8n/data:/home/node/.n8n
    - ../n8n/flujo.json:/etc/n8n/flujo.json:ro
    - ../n8n/init-n8n.sh:/etc/n8n/init-n8n.sh:ro
  entrypoint: ["tini", "--", "/bin/sh", "/etc/n8n/init-n8n.sh"]
  depends_on:
    - app

volumes:
  postgres_data:

```

Nota sobre seguridad de la configuración. La definición de credenciales estáticas y tokens en texto plano (POSTGRES_PASSWORD=security_pass y api.key=12345) constituye una decisión técnica orientada a facilitar el despliegue del laboratorio controlado, en concordancia con los alcances definidos en el capítulo 9. Por el contrario, las claves de servicios de inteligencia artificial (IA_API_KEY, GEMINI_API_KEY, groq_key) se gestionan mediante interpolación de variables de entorno del host (\${VAR:-}), garantizando la protección de secretos de producción fuera del entorno local.

A.3 Archivo Dockerfile

```
# Imagen base oficial de Python
FROM python:3.11-slim

# Evita archivos .pyc y buffers raros
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

# Instalamos dependencias del sistema necesarias
RUN apt update && apt install -y \
    wget \
    ca-certificates \
    && rm -rf /var/lib/apt/lists/*

ARG FFUF_VERSION=2.2.1
ARG FFUF_URL=https://github.com/ffuf/ffuf/releases/download/v${FFUF_VERSION}

RUN wget \
    ${FFUF_URL}/ffuf_${FFUF_VERSION}_linux_amd64.tar.gz \
    && tar -xzf ffuf_${FFUF_VERSION}_linux_amd64.tar.gz \
    && mv ffuf /usr/local/bin/ffuf \
    && chmod +x /usr/local/bin/ffuf \
    && rm ffuf_${FFUF_VERSION}_linux_amd64.tar.gz

# Instalamos SQLMap (clonamos el repo oficial)
RUN apt update && apt install -y git \
    && git clone --depth 1 https://github.com/sqlmapproject/sqlmap.git /opt/sqlmap \
    && rm -rf /var/lib/apt/lists/*

# Directorio de trabajo dentro del contenedor
WORKDIR /app

# Copiamos requirements primero (optimiza cache)
COPY requirements.txt .

# Instalamos dependencias Python
RUN pip install --no-cache-dir -r requirements.txt

# Copiamos el resto del proyecto
COPY . .

# Comando por defecto
CMD ["uvicorn", "api:app", "--host", "0.0.0.0", "--port", "8000"]
```

Anexo B — Fragmento Representativo: Pipeline de Ejecución

```
def run_security_pipeline(target_url, nivel="medium", cookies=None,
                        sqlmap_level="basic", progress_callback=None):

    init_db()
    ...
    if not cookies:
        cookies = establecer_sesion_automatica(target_url)
    limpiar_sesion_zap()
    if cookies:
        configurar_autenticacion(cookies)

    # --- 1. ZAP SPIDER ---
    spider_id = iniciar_spider(target_url)
    esperar_spider(spider_id, progress_callback=cb_spider)
    spider_urls = obtener_urls(spider_id)

    # --- 2. FFUF (antes del Active Scan) ---
    ffuf_raw = run_ffuf(target_url, WORDLIST_PATH, OUTPUT_DIR, cookies=cookies)
    ffuf_data = {} if ffuf_raw.get("skipped") else json.load(open(ffuf_raw["output_file"]))
    rutas_ffuf_nuevas = ffuf_data.get("results", [])

    # --- 3. INYECTAR RUTAS DE FFUF EN ZAP ---
    if rutas_ffuf_nuevas:
        agregar_urls_a_zap(rutas_ffuf_nuevas)

    # --- 4. ZAP ACTIVE SCAN ---
    ascan_id = iniciar_escaneo_activo(target_url)
    esperar_escaneo_activo(ascan_id, progress_callback=cb_ascan)
    reporte_zap_crudo = obtener_reporte_json()

    # --- 5. SQLMAP (sesión refrescada, URLs combinadas) ---
    lista_urls_total = list(set(spider_urls.get("results", []) +
                               [r.get("url", "") for r in rutas_ffuf_nuevas]))
    sqlmap_raw = run_sqlmap_batch(lista_urls_total, cookies=cookies,
                                sqlmap_level=sqlmap_level)

    return {"target": target_url, "spider_raw": spider_urls,
            "zap_raw": reporte_zap_crudo, "sqlmap_raw": sqlmap_raw,
            "ffuf_raw": ffuf_data}
```

Anexo C — Fragmento Representativo: Parser de ZAP

```
def parsear_zap(dato_dict_crudo):
    try:
        data = dato_dict_crudo
        vistas = set()
        alertas_parseadas = []
        for sitio in data.get("site", []):
            for alerta in sitio.get("alerts", []):
                for instancia in alerta.get("instances", []):
                    url = instancia.get("uri", "")
                    clave = (url, alerta.get("alert", ""))
                    if clave not in vistas:
                        vistas.add(clave)
                        alertas_parseadas.append({
                            "url": url,
                            "vulnerabilidad": alerta.get("alert", "Vulnerabilidad sin
nombre"),
                            "severidad": alerta.get("riskdesc", "No clasificado"),
                            "metodo": instancia.get("method", "N/A"),
                            "solucion": alerta.get("solution", "No hay solucion disponible")
                        })
        return {"herramienta": "ZAP", "total_alertas": len(alertas_parseadas),
                "alertas": alertas_parseadas}
    except (KeyError, TypeError) as e:
        print(f"Error al parsear los datos de ZAP: {e}")
        return {}
```

Anexo D — Ejemplo del JSON Unificado Final (corrida del 5 de agosto de 2026)

El siguiente extracto reproduce la estructura canónica del bloque resumen del esquema JSON unificado generado por el módulo de consolidación del pipeline para la ejecución documentada en el capítulo 13 (corrida del 5 de agosto de 2026), cuyos valores concuerdan íntegramente con los documentados en `reporte_seguridad.md`. Se presenta con fines de especificación de la arquitectura de interoperabilidad homogénea (validando la hipótesis H3), dado que los volcados crudos generados en `output/raw/` no se preservaron en el control de versiones debido a las reglas de exclusión del repositorio.

```
{
  "resumen": {
    "total_urls_unicas": 34,
    "urls_spider": 24,
    "alertas_zap": 37,
    "rutas_ffuf": 8,
    "vulnerabilidades_sqlmap": 4
  },
  "zap": {
    "herramienta": "ZAP",
    "host": "dvwa",
    "total_alertas": 37,
    "alertas": [
      {
        "url": "http://dvwa/about.php",
        "vulnerabilidad": "Content Security Policy (CSP) Header Not Set",
        "severidad": "Medium (High)"
      }
    ]
  },
  "ffuf": {
    "herramienta": "ffuf",
    "total_rutas": 8,
    "rutas": [
      {
        "url": "http://dvwa/setup.php",
        "status": 200
      }
    ]
  },
  "sqlmap": {
    "herramienta": "SQLMAP",
    "vulnerabilidades": [
      {
        "url": "http://dvwa/vulnerabilities/brute/?Login=Login&password=ZAP&username=ZAP",
        "salida": "... Type: boolean-based blind ... parameter 'username' is vulnerable"
      }
    ]
  }
}
```

Todas las rutas reportadas usan `http://dvwa/`, el hostname real de la red interna de Docker, en lugar del hostname ilustrativo `target.com` que figuraba en versiones anteriores del documento; esta ejecución corresponde a datos reales y no a un ejemplo construido.

Anexo E — Fragmento Representativo: Autenticación Automática

```
def establecer_sesion_automatica(target_url):
    """Login automatico en DVWA; reintenta inicializando la BD si falla."""
    session = requests.Session()
    def intentar_login():
        r = session.get(login_url, timeout=10)
        token = re.search(r"user_token['\'] value=['\']([^\']*)", r.text)
        session.post(login_url, data={"username": "admin", "password": "password",
                                     "Login": "Login", "user_token": token.group(1) if token
                                     else ""})
        r_sec = session.post(security_url, data={"security": "low", "seclev_submit":
        "Submit"})
        return "logout" in r_sec.text.lower()

    if intentar_login():
        return "; ".join(f"{k}={v}" for k, v in session.cookies.get_dict().items())

    # Login fallo: inicializar la base de datos de DVWA y reintentar
    session.post(setup_url, data={"create_db": "Create / Reset Database"})
    time.sleep(3)
    session.cookies.clear()
    if intentar_login():
        return "; ".join(f"{k}={v}" for k, v in session.cookies.get_dict().items())
    return None
```

Anexo F — Fragmento Representativo: API REST

```

app = FastAPI(title="Orquestador de Seguridad API", version="1.0.0")

class ScanRequest(BaseModel):
    target: str
    nivel: Optional[str] = "medium"
    cookies: Optional[str] = None
    callback_url: Optional[str] = None
    sqlmap_level: Optional[str] = "basic"

@app.post("/scan", status_code=202)
def iniciar_escaneo(request: ScanRequest, background_tasks: BackgroundTasks):
    scan_id = str(uuid.uuid4())
    create_scan(scan_id, request.target)
    background_tasks.add_task(ejecutar_pipeline_segundo_plano,
                              scan_id=scan_id,
                              target=request.target, nivel=request.nivel,
                              cookies=request.cookies, sqlmap_level=request.sqlmap_level)
    return {"message": "Escaneo lanzado", "scan_id": scan_id}

@app.get("/scan/{scan_id}/progress")
def get_scan_progress(scan_id: str):
    if scan_id in scan_progress_store:
        return scan_progress_store[scan_id]
    scan_data = get_scan_by_id(scan_id)
    if not scan_data:
        raise HTTPException(status_code=404, detail="Scan no encontrado")

    return {"percentage": 100, "message": "Análisis Completado"} if scan_data["status"] ==
"completed" \
        else {"percentage": 0, "message": "Iniciando o retomando análisis..."}

```

Anexo G — Frontend: Panel de Control

El frontend, desarrollado en React y TypeScript, consume la API REST del orquestador y ofrece dos experiencias de interfaz que coexisten en el proyecto: un panel de control con estética moderna (glassmorphism, gráficos interactivos) y una interfaz alternativa que recrea el escritorio de Windows 98, con sus propios íconos de escritorio, menú de inicio y una terminal estilo “MS-DOS Prompt” para seguir el progreso del análisis.

Motivación de la elección de diseño. La estética retro no es un tema visual accesorio, sino una decisión de diseño consciente del equipo, adoptada por tres razones concretas. Primero, una razón de identidad: distingue al proyecto de la generalidad de dashboards de seguridad con apariencia corporativa genérica, dándole una impronta propia reconocible dentro de un trabajo académico. Segundo, una razón de contexto cultural: el lenguaje visual de fines de los años noventa está asociado, dentro de la comunidad de seguridad informática, a una estética de herramienta técnica y de acceso directo a la información, coherente con la naturaleza ofensiva de las herramientas que el sistema orquesta. Tercero, una razón de alcance de aprendizaje: construir una interfaz sobre una paleta y unos patrones de interacción muy distintos a los de un framework de diseño moderno (ventanas arrastrables, íconos de escritorio, una terminal simulada) implicó resolver problemas de UI que no se presentan en un dashboard convencional, ampliando el trabajo de frontend del equipo más allá de componentes estándar.

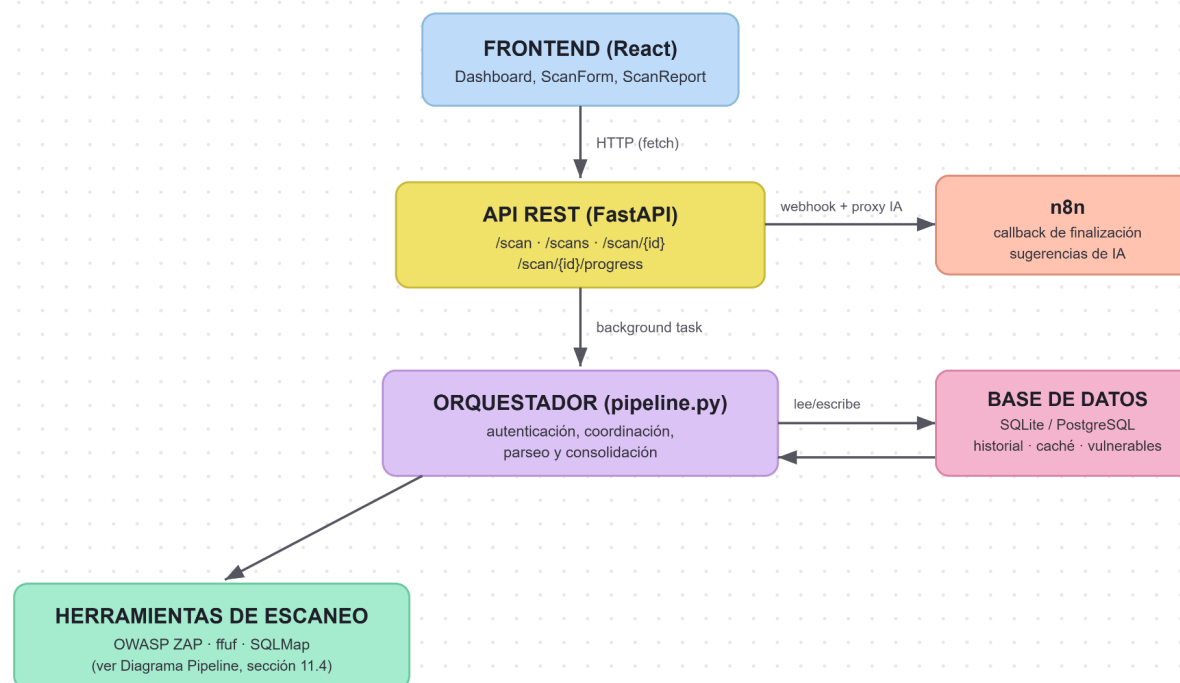
Panel de estado del análisis (DomainsDashboard). Muestra el objetivo escaneado como una tarjeta con su nivel de riesgo, la fecha del último análisis y su estado actual, consultado mediante polling al endpoint `/scan/{id}/progress`. Por las razones legales expuestas en el capítulo 16, el formulario de escaneo restringe el campo de URL objetivo, dejándolo deshabilitado y fijo en `http://dvwa`; el diseño multi-dominio queda como una capacidad arquitectónica disponible para habilitarse en el futuro, condicionada a un flujo formal de autorización.

Sugerencias de mitigación asistidas por IA (useAiAnalysis). Desde la tabla de vulnerabilidades, el usuario puede solicitar una sugerencia de mitigación para un hallazgo puntual; el frontend envía sus datos al endpoint `/api/n8n/analyze`, que delega el análisis a un flujo de n8n. Esta implementación reemplaza la propuesta original de “Idea Complementaria”: no existe un motor de priorización de riesgo basado en pesos configurables ni una versión con suscripción paga orientada a terceros —esa dimensión comercial se descarta explícitamente—, y el análisis multi-objetivo que la idea original imaginaba queda limitado, por diseño y por las restricciones legales del capítulo 16, a un único entorno de laboratorio autorizado (DVWA).

Anexo H — Diagramas en Tamaño Ampliado

Este anexo reproduce a mayor tamaño, para su mejor observación, las Figuras 1 a 4 ya presentadas y comentadas en las secciones 11.1, 11.2, 11.4 y 11.5: Figura 1 (Diagrama de Arquitectura de Servicio), Figura 2 (Diagrama Orquestador-Seguridad), Figura 3 (Diagrama de Pipeline de ejecución) y Figura 4 (Diagrama de secuencia UML — ciclo de vida de POST /scan).

Diagrama de Arquitectura de Servicio



Referencia

1. El Frontend lanza un escaneo (POST /scan) y consulta progreso/resultados (GET /scan/{id}, /scan/{id}/progress).
2. La API ejecuta el Orquestador como tarea en segundo plano (no bloquea la respuesta HTTP).
3. El Orquestador coordina ZAP/ffuf/SQLMap (ver Diagrama Pipeline) y lee/escribe caché e historial en la Base de Datos.
4. Al finalizar, la API notifica a n8n por webhook; el mismo canal se usa para proxyar sugerencias de mitigación con IA.

Figura 1 (ampliada). Diagrama de Arquitectura de Servicio. Elaboración propia.

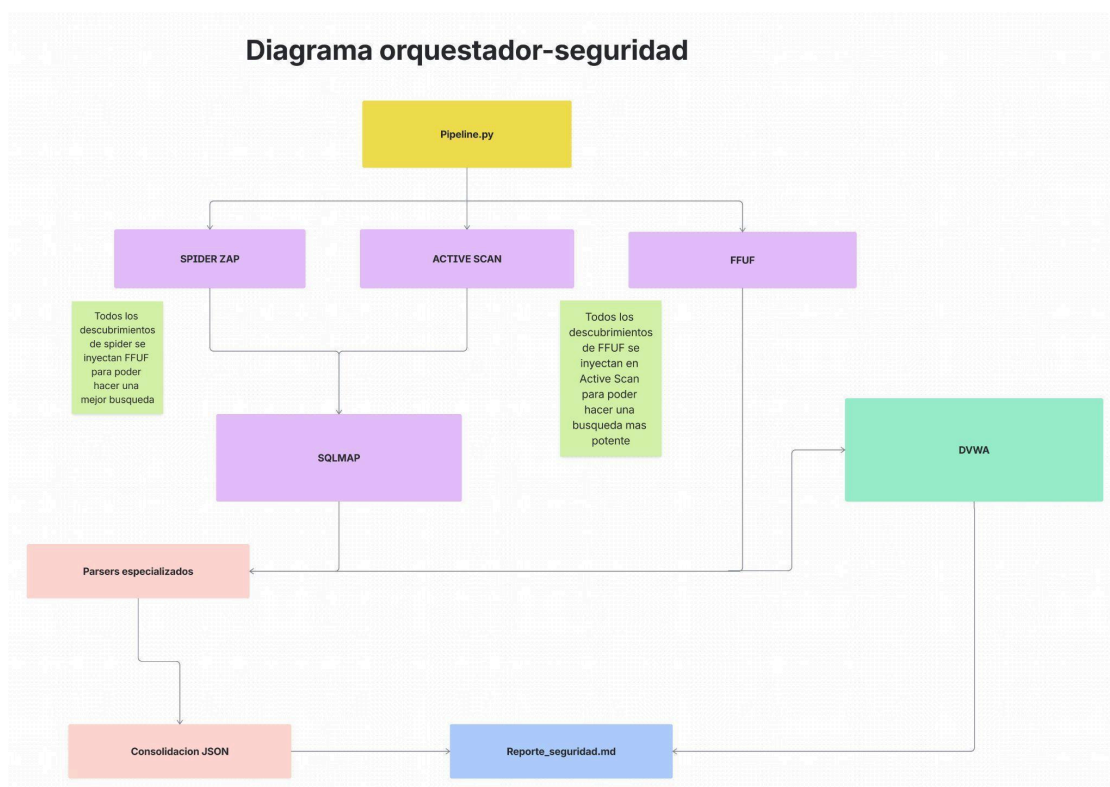


Figura 2 (ampliada). Diagrama Orquestador-Seguridad. Elaboración propia.

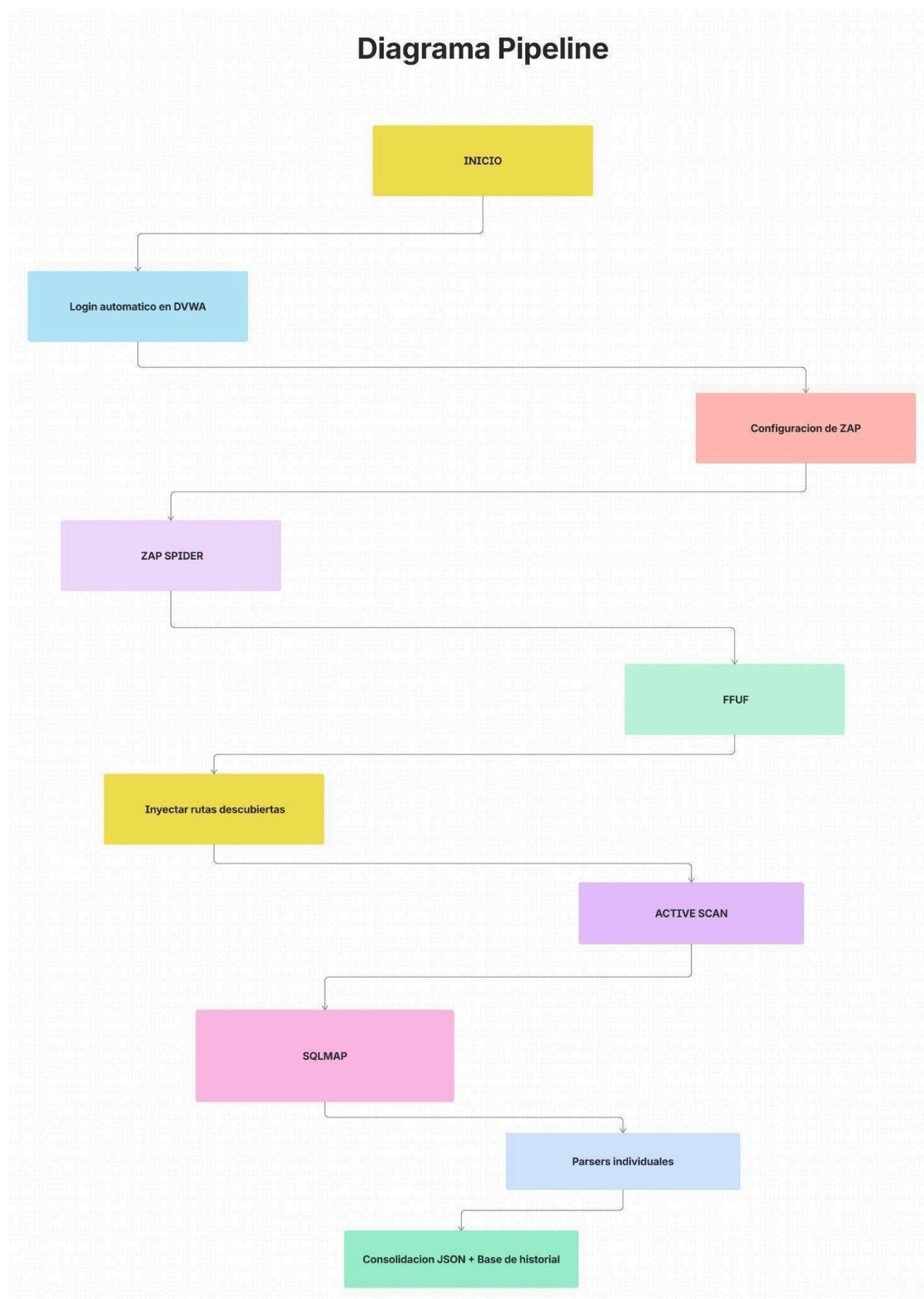


Figura 3 (ampliada). Diagrama de Pipeline de ejecución. Elaboración propia.

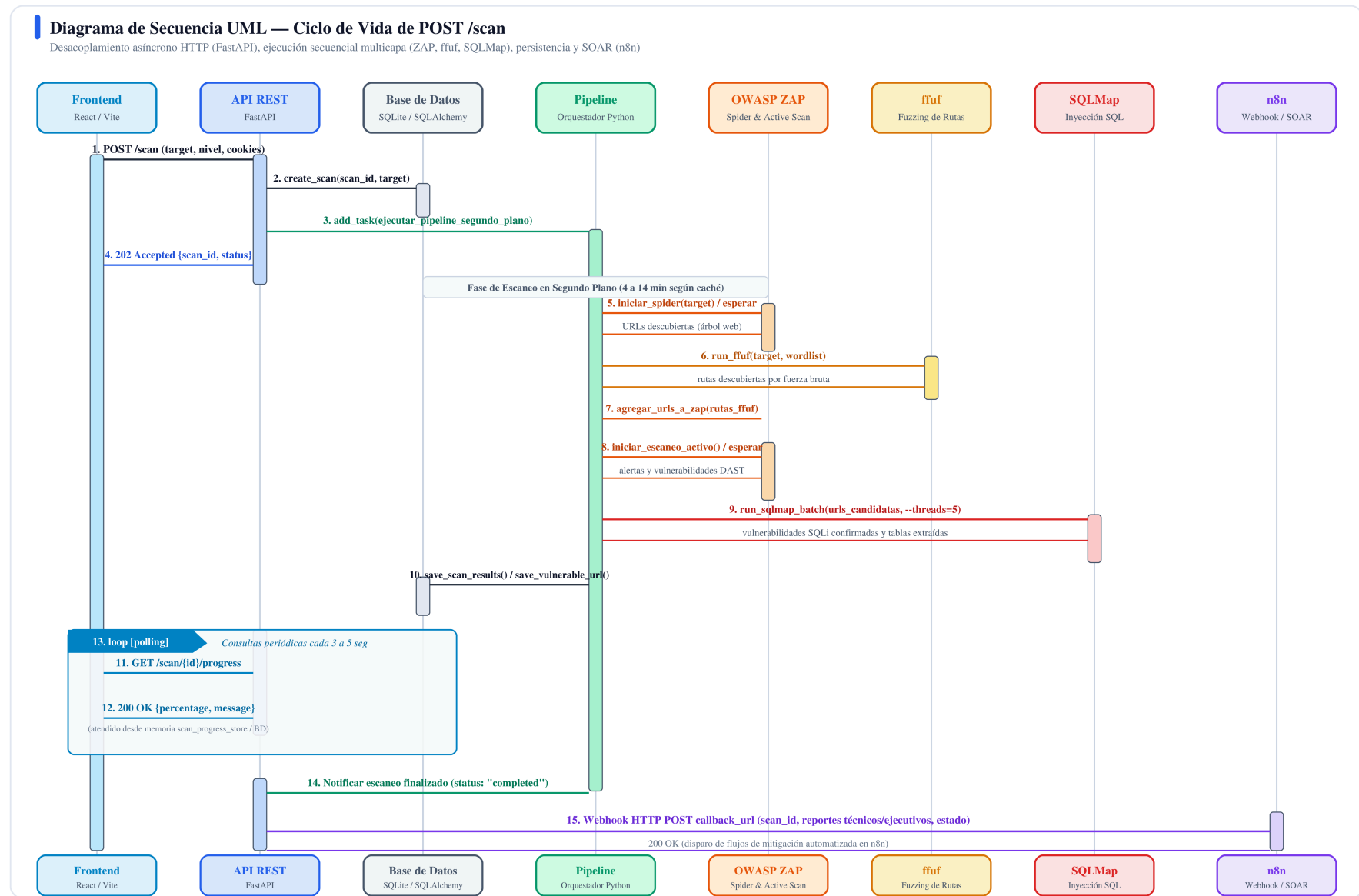


Figura 4 (ampliada). Diagrama de secuencia UML — ciclo de vida de POST /scan. Elaboración propia.

Anexo I — Disponibilidad de Código Fuente y Recursos Audiovisuales

Con el propósito de garantizar la reproducibilidad científica, la transparencia metodológica y la verificabilidad empírica del presente trabajo final de graduación, se ponen a disposición del tribunal evaluador de la UTN Facultad Regional Mendoza y de la comunidad académica los recursos digitales que sustentan el prototipo funcional desarrollado y su correspondiente validación en tiempo real.

I.1 Repositorio de Código Fuente y Prototipo Funcional (GitHub)

El código fuente integral del orquestador, los parsers sintácticos de seguridad, las configuraciones de contenerización con Docker Compose, los scripts de automatización y el panel de control web retro se encuentran alojados y versionados en el repositorio público de GitHub:

Repositorio oficial: github.com/tomimastro069/Tesis-final
Rama principal evaluada: `main`
Licencia de software: MIT License
Entorno de ejecución: Docker Engine 24+ y Docker Compose v2+

Instrucciones de clonación y despliegue rápido del entorno:

```
# Clonación del repositorio oficial
git clone https://github.com/tomimastro069/Tesis-final.git
cd Tesis-final
cd orquestador-seguridad

# Despliegue de la infraestructura multicontenedor (DVWA, ZAP, App, DB, n8n)
docker compose up -d --build
```

I.2 Registro Audiovisual de Demostración y Pruebas Empíricas (YouTube)

Aprobado por la Dirección del Trabajo Final, se elaboró un registro audiovisual demostrativo formal con una duración exacta de 9:00 minutos en resolución de alta definición (1080p FHD). El material documenta de manera continua el despliegue del entorno, la ejecución autónoma del pipeline secuencial de escaneo (ZAP, ffuf y SQLMap en modo de extracción completa `full_dump` y nivel de seguridad `low`), la interacción con el asistente de remediación basado en inteligencia artificial y la validación empírica del ahorro temporal provisto por el motor de caché incremental. Dicha demostración refleja una sesión independiente grabada el 15 de septiembre de 2026 (18:33 a 18:46 para la corrida basal y 19:15 a 19:18 para la optimizada) y publicada el 25 de septiembre de 2026, cuya concordancia y diferencias horarias respecto al benchmark formal del 6 de agosto (Tabla 10) se analizan en la sección 14.3.

Acceso al video demostrativo: youtube.com/watch?v=UA0rolg9ZKI
Título de la producción: Orquestador de Seguridad: Fuzzing Automatizado de Aplicaciones Web
Duración total: 09:00 minutos exactos | **Resolución:** 1080p (Full HD)
Institución: Universidad Tecnológica Nacional — Facultad Regional Mendoza

A continuación, se detalla la estructura narrativa y el desglose visual por bloques cronológicos que componen la demostración audiovisual:

Tabla 14. Estructura narrativa, marcas temporales y desglose visual del video demostrativo

Bloque y Tiempo	Apartado Visual en Pantalla	Descripción Técnica y Contenido Demostrado
Bloque 1 00:00 – 00:50 (50 s)	Pantalla principal de la interfaz de usuario retro (estilo Windows 98) con el asistente interactivo « <i>Acerca del Proyecto</i> » desplegado. El cursor recorre sucesivamente las pestañas: <i>Bienvenida</i> , <i>El Proyecto</i> , <i>Cómo Funciona</i> y <i>Tecnología</i> .	Presentación formal del proyecto ante la UTN FRM. Planteo de la problemática del pentesting web manual, justificación técnica del orquestador y presentación del stack tecnológico integrando OWASP ZAP, ffuf y SQLMap en contenedores Docker coordinados por Python.
Bloque 2 00:50 – 02:20 (1 min 30 s)	Apertura de la ventana « <i>Analizador de Seguridad</i> ». Selección de la opción « <i>Limpiar caché</i> », nivel de análisis « <i>Profundo</i> » y SQLMap en « <i>3 - Extracción Completa</i> ». Al accionar « <i>Comenzar Análisis</i> », emerge la consola <i>MS-DOS Prompt</i> mostrando el flujo de registros de escaneo acelerado a 16 ×.	Ejecución del primer escaneo completo sin caché (13 minutos de duración real: 18:33 a 18:46). Demostración del flujo: autenticación automatizada en DVWA con obtención de cookie de sesión, rastreo con Spider de ZAP, fuzzing de directorios con ffuf e inyección dinámica de rutas hacia ZAP activo y SQLMap.
Bloque 3 02:20 – 04:50 (2 min 30 s)	02:20: <i>Explorador de Reportes</i> y ventana <i>System Properties</i> con tarjeta verde de estado. 03:00: Pestaña <i>Device Manager</i> con el listado exhaustivo de vulnerabilidades por severidad. 03:45: Inspección de vulnerabilidad, selección de pestaña <i>AI Analysis</i> y activación del botón « <i>Analyze with AI</i> ». 04:10: Pestaña <i>Performance</i> con gráficos de barras. 04:30: Pestaña <i>ODBC Data Sources</i> con las tablas <i>dvwa.users</i> y <i>dvwa.guestbook</i> .	Consolidación y auditoría de resultados: resumen métrico de 59 URLs analizadas (54 descubiertas por Spider, 48 alertas de ZAP y 8 hallazgos de SQLMap). Clasificación de criticidad en Device Manager, generación interactiva de guías de remediación con fragmentos de código mediante IA y comprobación de la extracción efectiva de datos relacionales con hashes MD5.
Bloque 4 04:50 – 06:40 (1 min 50 s)	Apertura de la utilidad de contraste <i>WinDiff - dvwa vs dvwa</i> . Visualización comparativa a doble panel: panel izquierdo (amarillo) correspondiente a la ejecución « <i>Con Caché</i> » y panel derecho (verde) al análisis basal « <i>Sin Caché</i> ».	Evaluación analítica del impacto del motor de caché: demostración de más de 207.000 palabras omitidas por ffuf al no detectar alteraciones en la superficie y descarte inteligente en SQLMap de URLs sin hallazgos previos re-testeando únicamente endpoints vulnerables, manteniendo 100% de coincidencia en vulnerabilidades críticas.
Bloque 5 06:40 – 08:20 (1 min 40 s)	06:40: Configuración de nuevo escaneo con la casilla de limpiar caché desmarcada. 07:00: Ejecución en consola acelerada (16 ×) con rótulo temporal: « <i>Análisis con caché: 19:15 - 19:18</i> ». 07:40: Apertura de reportes con la tarjeta amarilla distintiva de análisis optimizado por caché.	Demostración empírica de reducción de tiempos en vivo: conclusión del pipeline completo en 3 minutos frente a los 13 minutos iniciales de la sesión grabada el 15 de septiembre de 2026, evidenciando una reducción temporal aproximada del 77% (10 minutos economizados) sin degradación de cobertura ni alteración de hallazgos, en consonancia cualitativa con el 68,9% documentado formalmente en la Tabla 10.
Bloque 6 08:20 – 09:00 (40 s)	Pantalla de cierre formal del proyecto con el logotipo oficial de la Universidad Tecnológica Nacional (UTN) y los datos institucionales del trabajo final.	Conclusiones finales sobre la viabilidad del orquestador SOAR liviano, balance costo-beneficio de la caché incremental y agradecimientos protocolares a los directores del trabajo final y al tribunal evaluador de la UTN FRM.

Nota. Elaboración propia a partir del guion técnico y registro audiovisual del proyecto (Mastropietro et al., 2026b).